

The Forensic Residual Framework

Evidence-First Software Construction, Behavioral Reconstruction,
and Deterministic Residual Endoduction from the DSFB Prior-Art Stack

Riaan de Beer

rdebeer.infinityabundance@gmail.com

GitHub: github.com/infinityabundance

ORCID: [0009-0006-1155-027X](https://orcid.org/0009-0006-1155-027X)

Version 1.0 — 2026

DOI: [10.5281/zenodo.22020499](https://doi.org/10.5281/zenodo.22020499)

Abstract

Building trustworthy software is not principally a production problem. It is an evidence problem. This is most visible when reimplementing mature software, where a new implementation may compile, pass its own tests, conform to a written standard, and improve memory safety while still differing from the implementation that users, scripts, network peers, filesystems, package managers, operators, and downstream programs actually depend upon. The same problem also appears in new software whenever tests encode developer expectation rather than measured behavioral evidence. Differences often occur at surfaces that are easy to dismiss as incidental: diagnostic wording, exit status, ordering, malformed-input behavior, filesystem side effects, timing boundaries, platform behavior, wire representation, undocumented defaults, and version-specific historical behavior.

This paper presents the Forensic Residual Framework (FRF), an evidence-first methodology for software construction, behavioral reconstruction, compatibility engineering, and verification-oriented development. FRF originates in the Drift-Slew Fusion Bootstrap (DSFB) prior-art stack, where residuals are treated as deterministic structural evidence rather than as noise to be averaged away. FRF treats observable disagreement between a declared authority and a candidate implementation as a residual that must be preserved and investigated before it may be normalized, dismissed, repaired, or accepted as an intentional divergence. Its central rule is residual primacy: a mismatch is not noise until evidence establishes why it is noise.

FRF organizes verification work as authority admission, behavioral hypothesis, oracle experiment, raw capture, implementation, forensic parity court, residual creation, deterministic residual endoduction, classification, minimization, correction or explicit explanation, regression invariant, reproducible receipt, and bounded claim. The framework introduces narrow executable courts, typed residual taxonomies, authority-admission records, evidence-preserving normalization, minimized reproducers, explicit negative capabilities, authority separation, claim-precision ladders, parity ledgers, replayable receipts, and anti-drift gates.

The contribution is methodological rather than a proof of universal equivalence. FRF provides a disciplined way to state exactly what has been observed, against which authority, under which conditions, what remains unknown, and what evidence would falsify the claim. Its objective is not to make software appear trustworthy. Its objective is to make trust inspectable.

Keywords: software verification; DSFB; deterministic inference; densorial inference; tekmeric inference; software reimplementation; compatibility testing; differential testing; test oracles; behavioral parity; reproducible software engineering; residual analysis; legacy software; evidence engineering; deterministic endoduction.

Prior-Art Notice

The core concepts in this paper are presented as an application and generalization of the author’s DSFB prior-art stack: deterministic drift–slew residual interpretation, residual envelopes, typed motifs, replayable evidence artifacts, non-claim ledgers, read-only observer surfaces, and generated claim discipline. FRF does not narrow that surface to administrative paper mechanics. It names a developer-facing method for applying the same residual-first discipline to software construction, compatibility work, debugging, auditing, and forensic parity courts.

The Taste Codex layer converts practical developer judgment into executable design constraints: invariants, courts, negative controls, residual fingerprints, and claim blockers. Its value in FRF is operational rather than explanatory.

Claim Hierarchy

This paper uses a deliberately conservative claim hierarchy.

Tier 1: Definitions and design rules.

The paper defines FRF objects such as authorities, courts, residuals, normalizers, receipts, deterministic endoduction maps, sensors, tekmeric claims, and bounded claims. These are conceptual and engineering-design claims.

Tier 2: DSFB-origin prior art.

The paper identifies the DSFB stack as the origin of FRF’s residual-first method: deterministic residual fields, drift–slew signatures, typed motifs, read-only observers, replay artifacts, non-claim ledgers, and generated evidence surfaces. The cited DSFB artifacts are prior-art substrate, not incidental inspiration.

Tier 3: Taste Codex developer judgment gates.

The Taste Codex layer defines operational gates for representation discipline, invariant-first design, unsafe and authority boundaries, compatibility repair, misuse-resistant APIs, performance grounding, simplicity accounting, and tooling that teaches. FRF expresses these as developer-operational artifacts rather than as aesthetic claims.

Tier 4: Evidence-motivated observations.

The paper reports recurring patterns observed across reconstruction projects: admitted authorities prevent guessed behavior from becoming claims; raw residual retention exposes compatibility defects; generated ledgers reduce prose drift. These observations motivate the framework but are not controlled experimental proof.

Tier 5: Open empirical questions.

The paper proposes metrics and research questions for future evaluation. It does not claim that FRF is universally superior to conventional development or rewrite workflows, only that it makes a specific evidence discipline explicit and falsifiable.

Load-bearing non-claim. FRF does not prove universal equivalence, legal clean-room status, security, standards correctness, or production replaceability. Deterministic endoduction, as used here, is a proposed fourth inference mode for coarsening residuals into typed evidence tokens from within a deterministic residual field; it complements rather than invalidates deduction, induction, and abduction. FRF supports bounded empirical claims over declared authorities, fixtures, environments, observables, and comparators.

Reader's Route

Problem framing.

Sections 1–8.

Operating kernel.

Section 2.

DSFB origin.

Sections 4–5.

Residual doctrine.

Sections 1 and 6.

Method.

Sections 9–13.

Developer design.

Sections 14–18 and 19–22.

Empirical evaluation design.

Sections 24–25.

Limits and threats.

Sections 26–28.

Reproducibility.

Appendix A, Appendix B, and Appendix C.

Contents

1	Introduction	5
1.1	Core Proposition	6
1.2	Contributions	6
2	Developer Operating Kernel	7
3	Full Mechanism Stack	7
4	DSFB Prior-Art Lineage	8
5	DSFB Principles Carried into FRF	9
6	Residual Primacy and Deterministic Endoduction	10
7	Problem Statement: Code Does Not Inherit Trust	12
8	Related Work and Positioning	13
9	Core Concepts	13
9.1	Authority and Admission	13
9.2	Observable	14
9.3	Court	14
9.4	Admissibility Envelope	14
9.5	Residual	14
9.6	Structured Unknown	14
9.7	Receipt	14

10 Formal Model	15
11 Workflow	15
11.1 Archaeology	16
11.2 Behavioral Hypothesis	16
11.3 Oracle Experiment	16
11.4 Raw Capture	16
11.5 Implementation and Court Execution	16
11.6 Residual Lifecycle	16
12 Worked Court: From Mismatch to Claim	16
13 Court Cookbook	18
14 Residual Taxonomy	19
15 Normalization as a Governed Operation	20
16 Authority Separation	20
17 Negative Capabilities	21
18 Developer Operating Rules	21
19 Developer Design	22
19.1 Invariant and Residual Corpora	22
19.2 Claim Compilation	23
19.3 Repository Shape	24
20 Practical Methods from Public Oracle Repositories	24
21 Flagship Case Files	25
21.1 Case 1: <code>zic-rs</code> and Authority Ecology	26
21.2 Case 2: <code>chrony-rs</code> and Host-Effect Isolation	27
21.3 Case 3: <code>gnucobol-rs</code> and Corpus Admission	27
21.4 Case 4: <code>ntpsec-rs</code> and Migration-Grade Parity	28
21.5 Case 5: <code>m4-rs</code> and Language-Surface Courts	28
21.6 Case 6: <code>ncurses-native</code> and Byte-Exact Terminal Envelopes	29
21.7 Case 7: <code>openntpd-rs</code> and Parity-Before-Label Gates	29
21.8 Reusable Flagship Pattern	30
22 The Taste Codex as Executable Developer Judgment	30
22.1 What the Taste Layer Adds	31
22.2 Repository Shape	31
22.3 Gate Record	32
22.4 Taste-to-FRF Translation	33
22.5 Design Gates	33
22.6 Worked Design Courts	34
23 Adoption Path	37
24 Empirical Evaluation Design	37
25 Quantitative Metrics	38

26 Limits of Equivalence Claims	39
27 Developer Failure Modes	39
28 Threats to Validity	40
29 Discussion	40
30 Conclusion	41
A Minimal Receipt Schema	42
B Developer Checklist	44
C Claude Skill Appendix: FRF Quick Reference	45
C.1 Suggested Skill Header	45
C.2 Part 1: Rosetta Table	45
C.3 Part 2: The Operating Core	47
C.4 Worked Example	48
C.5 When Not To Use This	49

1 Introduction

Software replacement is usually described in terms of implementation: understand an existing system, reproduce its features in another language or architecture, test the result, and migrate users. This description hides the hardest part of the problem. Mature software exposes far more behavior than its documented feature set.

A long-lived program accumulates an observable contract through years of use. Some of that contract is intentional. Some is historical accident. Some is implied by standards. Some is encoded only in tests. Some survives in downstream workarounds. Some depends on operating-system behavior. Some is visible only on malformed input. Some emerges from implementation details that no maintainer would deliberately specify but that users nevertheless observe.

A replacement can therefore be architecturally superior while remaining operationally incompatible. A command-line utility may differ in option permutation, stdout versus stderr routing, whitespace, locale, buffering, exit codes, signal handling, temporary-file behavior, path resolution, diagnostic ordering, environment variables, malformed byte sequences, and behavior under resource exhaustion. A network daemon adds packet bytes, retry timing, state-machine transitions, privilege boundaries, control protocols, kernel interfaces, persisted state, service-manager behavior, and peer interoperability.

Passing a conventional test suite is evidence about some of these surfaces. It is not evidence about all of them. The problem becomes acute when a replacement is intended to be a drop-in substitute for mature infrastructure. A phrase such as “compatible with” or “equivalent to” is compact in prose but contains a large empirical claim. It does not specify which reference version is authoritative, which environments were measured, which outputs were compared, whether malformed cases were included, whether a discrepancy was normalized away, or whether an independent system has consumed the resulting artifacts.

FRF starts by refusing this compression. Compatibility is treated as a collection of bounded claims over observable surfaces. Each claim must identify the evidence that supports it, and each artifact must state what it does not establish.

The same discipline applies before software becomes legacy. A new API, storage engine, service, model adapter, parser, or build tool begins accumulating behavioral promises as soon as another system depends on it. If those promises are not tied to admitted authorities, raw

observations, residuals, and receipts, they become folklore. FRF is therefore not only a method for recovering old contracts. It is a method for preventing new contracts from becoming unverifiable.

1.1 Core Proposition

The central object of FRF is the residual. If reference behavior O_R and candidate behavior O_C are observed under the same experiment and differ, the difference is preserved:

$$r = (O_R, O_C).$$

The notation is conceptual rather than necessarily numerical. A residual may be a changed packet byte, an extra diagnostic, a different ordering, a missing file, an altered process transition, an exit-code discrepancy, a timing difference, or an unexpected performance characteristic.

FRF then asks why the residual exists. A mismatch that is fixed becomes useful evidence. A mismatch shown to arise from nondeterministic transaction identifiers becomes useful evidence. An intentional safer behavior that differs from the reference becomes useful evidence if its scope is explicit. An oracle bug, harness bug, or unresolved mismatch remains useful because it marks the boundary beyond which the project may not claim equivalence.

Core proposition. The unexplained remainder is often more informative than the aggregate pass rate.

1.2 Contributions

This paper contributes:

1. A terminology and object model for evidence-first software construction and behavioral reconstruction.
2. An explicit software-development formulation of the DSFB prior-art stack’s residual-first method.
3. A residual lifecycle that preserves disagreement before explanation or normalization.
4. A deterministic endoduction step that coarsens raw residuals into typed evidence tokens without smoothing them away.
5. A court-and-receipt design for replayable compatibility and behavioral claims.
6. A claim-governance model separating evidence axes, authorities, non-claims, and unknowns.
7. A developer-facing scaling design: court registries, residual ledgers, residual corpora, invariant banks, receipt indices, claim ledgers, anti-drift gates, witness maps, residual token grammars, and court cookbooks.
8. A developer operating kernel and full mechanism stack showing how DSFB, FRF, and Taste Codex artifacts interlock in ordinary repositories.
9. Flagship case-file patterns from reconstruction repositories, each stated as bounded evidence machinery rather than universal proof.
10. A set of empirical research questions and metrics for evaluating whether FRF improves verification quality in future controlled studies.

2 Developer Operating Kernel

The full framework can scale into ledgers, matrices, corpora, receipts, gates, and cross-version courts. The daily operating kernel is smaller. A developer should be able to remember it after one read:

1. Admit the authority.
2. Declare the claim.
3. Name the court.
4. Capture raw observations.
5. Preserve residuals.
6. Coarsen residuals into tokens.
7. Route tokens to courts.
8. Add negative controls.
9. Emit receipts.
10. Compile claims and non-claims.

This kernel is not a summary that replaces the rest of the paper. It is the execution spine. Every larger mechanism in FRF exists to make one of these steps more precise, replayable, falsifiable, or resistant to claim drift.

3 Full Mechanism Stack

FRF is intentionally a stack, not a single testing trick. Its value comes from keeping several mechanisms distinct while making them compose.

Layer	Mechanisms	Developer artifact	Claim effect
DSFB residual substrate	Residual primacy, drift-slew signs, residual envelopes, deterministic endoduction, densorial residual fields.	Residual token grammar, admissibility envelope, residual trajectory ledger.	Makes disagreement interpretable rather than disposable.
FRF behavioral evidence	Authority admission, courts, raw capture, normalization governance, minimization, receipts.	Court manifests, captures, residual ledger, verdict case file.	Bounds behavioral and compatibility claims.
Taste Codex gates	Invariant ledgers, representation gates, boundary gates, API misuse gates, compatibility repair gates, performance envelopes.	Gate records, negative controls, residual fingerprints, claim blockers.	Blocks design-quality overclaims.
Claim compiler	Receipts, non-claims, blocked claims, freshness gates, witness maps.	Generated claim ledger and non-claim ledger.	Prevents prose from exceeding evidence.

Layer	Mechanisms	Developer artifact	Claim effect
Empirical program	Replay rates, mutation kill rates, transfer rates, closure latency, observability escapes.	Metrics dashboard and evaluation campaigns.	Makes framework performance measurable.

The stack is deliberately not collapsed. A green behavioral court does not satisfy a Taste Codex gate. A clean Taste Codex gate does not establish protocol parity. A DSFB residual token does not by itself license a public claim. The claim becomes admissible only when the relevant layers produce receipts whose scope covers the sentence being emitted.

4 DSFB Prior-Art Lineage

FRF is not presented as an isolated testing convention. Its conceptual origin is the DSFB research and implementation stack, where residuals are treated as structured deterministic evidence. The relevant transfer is direct: DSFB turns residual products into typed structural episodes; FRF turns behavioral mismatches into typed forensic residuals that drive parity courts.

Several public DSFB artifacts establish the practical pattern. The DSFB core specification defines deterministic bounded disturbances, residual decomposition into drift and slew, residual envelopes, update semantics, and implementation-aligned reproducibility conventions without Gaussian noise, covariance matrices, channel independence assumptions, or probabilistic optimality criteria [14]. *Residual Trajectories as Primary Epistemic Objects* states the key conceptual inversion: the residual trajectory, not only the converged endpoint, can carry the most important structural information [15]. *Alternative Deterministic Structural Inference* organizes the broader stack through core theorems, component theorem banks, structural realization results, and a deterministic compositional inference algebra; for FRF, this becomes invariant-bank discipline rather than ad hoc test accumulation [16]. The Deterministic Disturbance Modeling Framework defines admissible disturbance classes and explicitly bounds its guarantees to deterministic envelope-admissible regimes [17]. `dsfb-debug` is described as a deterministic, read-only augmentation layer that reads residuals from observability tools and code-defect catalogs, computes drift, slew, grammar state, motif, and episode, and produces deterministic reproducible outputs on byte-stable inputs [18]. `dsfb-forensics` wraps the observer with a deterministic forensic engine that records residuals, trust weights, trust-gated causal graph fragmentation, shatter events, silent failures, and timestamped audit artifacts [19]. `dsfb-database` applies the same residual-trajectory grammar to SQL telemetry as a read-only sidecar, with explicit non-claims, deterministic reproduction commands, replay invariants, and provenance manifests [20]. `dsfb-gray` applies the pattern to Rust crate auditing: source structure is converted into deterministic findings, assurance signals, audit traces, and attestation artifacts, while its README states that public metrics should be regenerated from code and artifacts rather than maintained by hand [21]. `dsfb-gpu-debug` adds a clear-box accelerator version of the same idea: residual signs, detector motifs, grammar, bank-governed episode admission, verdict case files, stage hashes, and explicit cross-architecture replay boundaries [22].

The common design is not merely “deterministic code.” Many methods called deterministic compute a fixed estimator while relying on probabilistic interpretation, likelihood assumptions, Gaussian residual models, learned weights, posterior updates, or stochastic calibration. DSFB uses *pure deterministic* in a narrower engineering sense: given byte-stable inputs, fixed configuration, and an admitted environment, the inference path is a deterministic mapping from residual field to typed structural evidence. No stochastic noise model is required to make the residual meaningful. No averaging step is allowed to erase the local structure before interpretation.

Densors and densorial inference. This paper uses *densor* for a deterministic tensor-like evidence object whose axes preserve residual identity, surface, time or order, channel, authority, envelope relation, drift, slew, trust, provenance, and disposition. A densor is not a random tensor and is not primarily a vehicle for statistical flattening. It is a structured residual field designed to keep coarseness visible. *Densorial inference* is inference over these deterministic residual fields: the system reasons over typed residual shape, envelope crossing, drift dwell, slew density, motif membership, authority conflict, and court disposition without converting the residual field into a probability distribution.

Tekmeric inference. This paper uses *tekmeric inference* for bounded inference from concrete evidence marks: captures, hashes, residual tokens, receipts, replay commands, non-claims, and court outcomes. A tekmeric claim is not stronger because it sounds plausible. It is stronger only when the evidence marks supporting it are more direct, replayable, independent, or complete over the declared surface. In FRF, tekmeric inference is the claim layer built on top of densorial residual interpretation.

Prior-art scope. The DSFB Zenodo and GitHub records cited here should be read as the prior-art substrate for FRF rather than as incidental related work [18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31]. FRF contributes a software-development and parity-court expression of that substrate. It does not claim that every DSFB hypothesis has been universally validated, and it does not claim that deterministic residual interpretation dominates probabilistic methods in all settings. It does claim that residual coarsening, replayable evidence, and bounded non-probabilistic claim discipline are practical developer tools.

5 DSFB Principles Carried into FRF

The DSFB papers strengthen FRF most when their concepts are translated into developer operations. The following principles are the practical inheritance.

DSFB principle	FRF developer translation
Residual trajectory primacy	Treat the behavioral mismatch history as the object of evidence, not merely the final pass/fail verdict. Preserve how the residual changes across commits, versions, fixtures, environments, and minimization steps.
Drift-slew sign tuple	Represent each residual with at least magnitude, persistence/drift, and abrupt-change/slew fields. A one-byte mismatch that persists for years and a one-time ordering permutation should not share the same evidence shape.
Admissibility envelope	Every court needs declared admissibility conditions: authority version, fixture family, environment, normalizers, timing bounds, platform, and comparison relation. Outside the envelope, the honest result is UNKNOWN or BLOCKED, not PASS.
Grammar state	Residuals should move through explicit states such as Admissible, Boundary, Violation, Recovery, Unknown, and Intentional Divergence. These states make the court graph executable.

DSFB principle	FRF developer translation
Structured Unknown	Unknown is not empty. A court may fail to name the cause while still emitting a complete residual sign, grammar state, provenance, and next-court route. This is endoductive value rather than detection failure.
Routed evidence	A reason code alone is insufficient. Residual evidence should be routed through surface-specific courts: text, exit, filesystem, wire, timing, state, security, process, and authority conflict.
Trace-event collapse	Developer review load matters. The framework should compress many raw diffs into fewer typed residual episodes while retaining links to the raw captures.
Semantic non-bypass	No high-level claim may be emitted directly from an accelerated score, detector count, or green test. Claims must pass through the declared grammar, bank, receipt, non-claim, and replay contract.
Verdict case file	The receipt should become a verdict case file: authority hash, fixture hash, environment digest, comparator hash, residual-token hash, stage hashes, claims, non-claims, and replay command.
Invariant bank	Useful residuals should accumulate into named invariants organized by surface, authority, envelope, and claim. This is the developer analogue of DSFB theorem-bank discipline.
Replay boundary	Deterministic replay is always scoped. Identical artifacts on a pinned toolchain do not imply cross-architecture identity unless cross-architecture courts have been run.

The mentor rule. Do not make FRF smaller than its origin. A narrow formulation as “careful compatibility testing” loses the practical novelty: residuals are a deterministic evidence language. Courts are not merely tests. They are the adjudication machinery that lets that language change software decisions.

6 Residual Primacy and Deterministic Endoduction

FRF is organized around residual primacy:

A mismatch is not noise until evidence establishes why it is noise.

This rule is stricter than ordinary differential testing. Ordinary comparison often treats residuals as failures to be removed, filtered, or explained informally. FRF treats the residual as the first-class artifact. The raw mismatch is preserved, assigned identity, coarsened into a typed evidence token, routed into further courts, and only then repaired, normalized, or bounded.

This paper uses *deterministic endoduction* for that coarsening step. Within the DSFB lineage, endoduction is proposed as a fourth practical inference mode: inference from within a structured residual field. This paper treats that fourth-mode framing as a philosophical hypothesis, not an established theory; the engineering mechanism below, κ as a deterministic coarsening function, does not depend on that hypothesis being correct. Deduction derives necessary consequences from rules. Induction estimates general patterns from observations. Abduction proposes explanatory hypotheses. Endoduction maps the interior structure of a

residual field into the system’s own typed evidence language without first turning that field into a probability distribution. In FRF, the local endoduction map is:

$$\kappa(r_{\text{raw}}) = (\textit{kind}, \textit{surface}, \textit{authority}, \textit{magnitude}, \textit{scope}, \textit{disposition}, \textit{next_court}).$$

The function κ is deterministic for a given schema version and residual input. It does not infer hidden truth, fit a probabilistic model, learn weights, or erase inconvenient structure. It coarsens raw disagreement into a stable token that code can route, count, replay, minimize, and use to select the next experiment. This is the point at which the residual becomes executable evidence.

Canonical loop. The shortest form of FRF is:

Authority $\rightarrow$ Court $\rightarrow$ Capture $\rightarrow$ Residual $\rightarrow$ Endoduction
 $\rightarrow$ Route $\rightarrow$ Disposition $\rightarrow$ Receipt $\rightarrow$ Claim.

This loop is intentionally small enough to implement in an ordinary repository. The authority licenses the question. The court runs the question. The capture preserves the observation. The residual preserves the disagreement. Endoduction turns the residual into a typed token. Routing selects the next court or blocker. Disposition records what became of the residual. The receipt binds the evidence. The claim is allowed only if its scope is covered by the receipt chain.

Coarsening, not smoothing. Smoothing makes residuals less visible by averaging, filtering, thresholding, or normalizing away local disagreement. It can then report that nothing remains to infer over, because the information-bearing remainder has been removed. FRF allows normalization only after raw preservation and only when the claim is weakened accordingly.

Coarsening is the opposite discipline. It is lossy only in a declared, typed, replayable way. Instead of averaging away a jagged residual field, endoduction maps it into a coarser but more legible grammar: drift, slew, boundary, violation, motif, authority conflict, disposition, next court. The coarseness is not a defect. It is the scale at which the residual becomes usable by developers and by deterministic tooling. The raw capture remains available; the coarsened token makes the evidence operational.

Densorial court driving. Forensic parity courts should not be driven only by a human reading a diff. They should be driven by densorial residual tokens. An exit residual routes to exit-class minimization. A text residual routes to diagnostic court refinement. A filesystem residual routes to side-effect capture. A timing residual routes to repeated-run and clock-domain courts. An authority-conflict residual routes to witness-separation review. The court graph is therefore not a manually curated checklist alone; it is a deterministic response to the residual language produced by endoduction.

For example, a raw stderr mismatch may be coarsened into:

```
kind: text
surface: diagnostic-routing
authority: reference-cli-v1.8.2
magnitude: first-line-token-change
scope: malformed-input
disposition: open
next_court: cli-diagnostic-minimization
```

This token does not claim to explain the mismatch. It makes the mismatch operational. A court runner can route it to minimization. A claim compiler can block claims about byte-identical diagnostics. A ledger can count diagnostic-routing residuals across versions. A developer can see what must be investigated next.

Residual token grammar. A practical implementation should make the coarsened residual language explicit. The following schema is illustrative rather than normative:

```

residual_token:
  schema_version: frf-token-v1
  id: string
  kind: text | exit | wire | state | timing | filesystem |
        security | platform | performance | harness | unknown
  surface: string
  authority: string
  court: string
  envelope_state: admissible | boundary | out_of_envelope
  sign:
    magnitude: none | small | structural | blocking | opaque
    drift: transient | persistent | recurrent | version_stratified
    slew: none | abrupt | gradual | burst
  grammar_state:
    observed | violation | recovery | intentional_divergence |
    authority_conflict | harness_fault | structured_unknown
  raw_capture:
    reference_hash: string
    candidate_hash: string
  disposition:
    open | fixed | normalized | explained | intentional |
    environmental | oracle_version | blocked | unknown
  route:
    next_court: string
    blocks_claims: [string]
    weakens_claims: [string]

```

The grammar is deliberately coarser than the raw diff. It does not replace raw bytes, logs, traces, packets, files, or fixtures. It gives those facts a stable shape that code review, CI, documentation generation, and claim compilation can use.

Why this matters for new software. In new software there may be no legacy executable to reconstruct. There are still authorities: protocol specifications, public API contracts, golden fixtures, property definitions, model checkers, customer-visible behavior, migration promises, security policies, persistence formats, and prior releases of the same product. FRF applies whenever a project wants claims to be earned by replayable evidence rather than developer expectation. Legacy reconstruction is the hard case because the authority already exists outside the new implementation; new software can benefit by creating its authorities before claims drift.

7 Problem Statement: Code Does Not Inherit Trust

Every nontrivial implementation begins with a trust deficit. For a rewrite, the deficit is compatibility with an incumbent. For new software, the deficit is correspondence between implemented behavior and the authority the project claims to satisfy: a specification, public contract, invariant, release promise, operational policy, persistence format, or declared safety boundary. Changing implementation language can remove classes of defects, but this says little about behavioral compatibility. Memory safety and behavioral equivalence answer different questions. A parser can be memory safe while rejecting input accepted by the reference. A

network daemon can avoid memory corruption while encoding one field incorrectly. A command-line utility can be written in a safer language yet emit subtly different diagnostics that break scripts. Conversely, a replacement may intentionally reject unsafe behavior the original accepts. That may be desirable, but it is a divergence, not parity.

FRF therefore separates evidentiary rungs:

memory safety $\nrightarrow$ behavioral parity,
 format validity $\nrightarrow$ reference compatibility,
 reference compatibility $\nrightarrow$ standards correctness,
 finite fixture match $\nrightarrow$ universal equivalence.

This prevents claim drift: the tendency for prose to become stronger than its evidence. FRF imposes an asymmetry: evidence may strengthen a claim, but prose may not strengthen evidence.

8 Related Work and Positioning

FRF sits within established software-testing practice. The test oracle problem asks how a testing process determines whether observed behavior is correct [1]. Differential testing presents comparable systems with the same inputs and treats disagreements as candidate defects [2]. Compiler testing work such as Csmith demonstrated that automatically generated programs can reveal serious implementation defects [3], while reduction tools such as C-Reduce made large failures tractable [4]. Network-verification research similarly uses generated inputs, comparisons with actual router software, discrepancy detection, and delta debugging [5].

FRF adopts these ideas rather than claiming to supersede them. Its contribution is a governance layer around long-lived reconstruction work. Differential testing asks whether systems disagree. Test reduction asks whether the disagreement can be made smaller. A test oracle asks against what expected behavior execution should be judged. FRF additionally asks which reference binary was admitted, which authority it possesses, what surfaces were measured, which raw values existed before normalization, what remains unexplained, which claim the experiment may support, and what stronger claim it explicitly may not support.

9 Core Concepts

9.1 Authority and Admission

An authority is the witness a claim appeals to. In reconstruction work it is often a reference implementation whose behavior is being measured. In new software it may be a protocol specification, public API contract, migration promise, golden fixture set, property definition, model checker, persisted-format schema, security policy, or prior release. An authority is authoritative only for a declared question. A historical executable may be authoritative for compatibility with that executable but wrong according to a modern standard. An RFC may be authoritative for protocol requirements but unable to reveal diagnostic text. A distribution package may establish what that distribution shipped but cannot establish behavior for every derived package.

An authority becomes usable only after admission. Admission records enough information to identify the thing whose behavior is being cited: name, version, build options, package version, executable path, cryptographic hash, source release, commit, architecture, operating system, dynamic dependencies, enabled features, and provenance where applicable. Admission prevents oracle drift, where a test described as matching a reference silently begins running against another release after a package update.

9.2 Observable

An observable is something an external observer can measure and that may belong to the compatibility contract: wire bytes, stdout, stderr, exit status, generated files, metadata, system calls, process topology, permissions, persisted state, protocol state, ordering, latency, retry behavior, logging, statistics, resource usage, security boundaries, or downstream-reader behavior.

9.3 Court

A court is a narrow differential experiment addressing one explicit compatibility question. A test named `full_compatibility` is weak as an evidentiary unit because success is easy to overinterpret and failure gives little diagnostic locality. A court such as “does the candidate reproduce reference DNS-name compression for this input family?” has a defensible scope.

A court should declare its question, admitted authorities, fixture set, environment, observed surfaces, comparison functions, permitted normalizations, expected invariants, and falsification condition.

9.4 Admissibility Envelope

An admissibility envelope is the declared boundary inside which a court is allowed to speak. In DSFB, envelopes specify deterministic residual regimes in which suppression, recovery, or interpretation claims are meaningful. In FRF, the envelope specifies the authority version, fixture family, environment, platform, observable axes, normalizers, timing assumptions, comparator relation, and replay conditions.

The envelope is not decorative. It prevents a local court from becoming a universal claim. If an execution falls outside the envelope, the correct court state is not PASS. It is OUT-OF-ENVELOPE, UNKNOWN, or BLOCKED, depending on what happened.

9.5 Residual

A residual is any observed difference between an admitted authority and an implementation within a declared comparison surface. Residuals are not synonymous with bugs. A residual is initially epistemically neutral: these observations differ. Later classification may identify an implementation defect, reference quirk, intentional divergence, environmental variation, version difference, harness error, nondeterminism, unsupported behavior, or unknown cause.

9.6 Structured Unknown

An unknown residual is not a blank. A structured Unknown preserves the residual sign, raw capture hashes, court envelope, observed grammar state, possible blocked claims, and next-court route even when no explanation or motif is yet admitted. This is the software-engineering value of endoduction: the project can act on organized residual structure before it can name the cause.

9.7 Receipt

A receipt is a durable record binding an evidentiary result to the conditions under which it was produced. It should identify the court, authority, implementation, fixture, environment, comparator, residual state, result, and reproduction procedure. Stronger receipts bind artifacts with cryptographic hashes.

A receipt is not merely a test report. It is a claim-provenance object.

10 Formal Model

Let the admitted executable authority be R , the candidate implementation be C , an input fixture be x , and a controlled environment be e . For non-executable authorities, R denotes the executable oracle derived from the authority: a validator, model checker, contract test, schema checker, reference vector, or other admitted witness. Let the observation function for surface a be

$$O_a(S, x, e),$$

where S is either R or C .

An observation vector is

$$O(S, x, e) = (O_{\text{wire}}, O_{\text{stdout}}, O_{\text{stderr}}, O_{\text{exit}}, O_{\text{filesystem}}, \\ O_{\text{state}}, O_{\text{ordering}}, O_{\text{timing}}, O_{\text{process}}, O_{\text{security}}, \dots).$$

The axes are court-specific.

For each observable axis a , a comparator

$$\Delta_a(O_a(R, x, e), O_a(C, x, e))$$

returns either equivalence under the declared comparison relation or a residual r_a .

A residual can be represented as

$$r_a = (id, court, axis, raw_R, raw_C, context, classification, explanation, reproducer, invariant).$$

The raw observations are immutable.

A court is

$$\mathcal{C} = (Q, A, X, E, \Omega, \Delta, N, I, F),$$

where Q is the narrow question, A the admitted authority set, X the fixtures, E the environment, Ω the observed surfaces, Δ the comparators, N the permitted normalizers, I the invariants, and F the falsification condition.

A receipt is

$$P = (\mathcal{C}, H_R, H_C, H_X, H_E, H_{\text{captures}}, Residuals, Result, Reproduction).$$

The claim-admission rule is:

$$Claim(K) \Rightarrow \exists P.$$

More strongly:

$$Scope(K) \subseteq Scope(P_1 \cup P_2 \cup \dots \cup P_n).$$

The prose claim may not cover a larger surface than the receipts collectively measure.

11 Workflow

The FRF workflow is:

$$\begin{aligned} & \text{archaeology} \rightarrow \text{behavioral hypothesis} \rightarrow \text{authority admission} \rightarrow \text{oracle experiment} \\ & \quad \rightarrow \text{raw capture} \rightarrow \text{implementation} \rightarrow \text{forensic parity court} \rightarrow \text{raw residual} \\ & \quad \rightarrow \text{deterministic endoduction} \rightarrow \text{classification} \rightarrow \text{minimization} \\ & \quad \quad \rightarrow \text{fix or explanation} \\ & \quad \rightarrow \text{regression invariant} \rightarrow \text{receipt} \rightarrow \text{bounded claim}. \end{aligned}$$

11.1 Archaeology

FRF begins before implementation. The engineer reconstructs source releases, interfaces, version changes, build options, platform branches, protocol versions, historical defects, downstream expectations, documentation, tests, issue history, external standards, and deployment practices. The objective is not merely to understand current code organization. It is to discover where compatibility can hide.

11.2 Behavioral Hypothesis

A useful hypothesis is falsifiable. “Implement configuration parsing” is weak. “Reference release R_v accepts fixture classes A–D, rejects E–G, exits with class 1 on malformed directives, and emits diagnostic class Y at the first malformed line” is testable.

11.3 Oracle Experiment

Before implementing candidate behavior, FRF prefers to observe the reference. This order reduces confirmation bias. If the replacement is written first, engineers tend to construct tests around behavior already implemented. If the oracle experiment comes first, the reference produces evidence that may contradict assumptions.

11.4 Raw Capture

The oracle’s output is captured before normalization or interpretation. Capture may include packets, streams, return status, files, metadata, syscall traces, state transitions, logs, performance counters, process trees, or downstream-reader observations. Interpretation can change; raw evidence should not.

11.5 Implementation and Court Execution

Implementation attempts to satisfy measured behavior. In reconstruction work, FRF does not require transliteration: the internal architecture may differ substantially from the reference as long as externally claimed behavior is demonstrated. In new software, the same rule applies to contracts, schemas, invariants, and specifications: implementation design is free, but claims remain bound to admitted authorities. Both authority witness and candidate are then executed or evaluated under the court and produce raw captures.

11.6 Residual Lifecycle

Every unexplained difference becomes a residual. Each residual is classified, minimized, corrected or explicitly explained, and converted into a durable invariant where possible. A bug that was difficult to discover once should become cheap to detect forever.

12 Worked Court: From Mismatch to Claim

This section gives a minimal court in the style FRF expects developers to implement. The domain is intentionally ordinary: command-line diagnostic behavior.

Question. For malformed input in fixture family X , does the candidate preserve the admitted reference’s exit class and first diagnostic line?

Authority admission.

```
authority:
  id: ref-cli-1.8.2
  kind: executable-reference
  version: 1.8.2
  executable_sha256: 91b7...c21a
  platform: x86_64-linux
```

Fixture.

```
argv: ["tool", "--strict", "fixtures/malformed-path.conf"]
stdin_sha256: e3b0...b855
fixture_sha256: a817...0e44
```

Raw capture.

```
reference.exit: 2
reference.stderr[0]: "tool: malformed-path.conf:4: unknown directive 'servre'"
candidate.exit: 1
candidate.stderr[0]: "error: unknown directive servre at line 4"
```

Residual. The court does not immediately decide that the candidate is wrong, that the reference is right, or that the wording is cosmetic. It persists two residuals:

```
residuals:
- id: cli-exit-0007
  kind: exit
  raw_reference: 2
  raw_candidate: 1
  disposition: open
- id: cli-text-0011
  kind: text
  surface: first-diagnostic-line
  disposition: open
```

Endoduction. The raw residuals are coarsened into typed evidence tokens:

```
tokens:
- exit/malformed-input/class-change/open -> court: cli-exit-minimize
- text/diagnostic-routing/token-change/open
  -> court: cli-diagnostic-minimize
```

The tokens do not erase the raw lines. They route the work. The exit token blocks any claim about malformed-input exit parity. The text token blocks byte-identical diagnostic claims and selects a minimization court.

Minimization. The original fixture is reduced to:

```
servre 127.0.0.1
```

The residual persists under the minimized fixture. The misspelled directive is therefore sufficient to expose the difference.

Resolution. Suppose the candidate is changed to preserve the reference exit class while intentionally retaining clearer diagnostic wording. The residual dispositions become:

```
cli-exit-0007: fixed
cli-text-0011: intentional-divergence
```

Receipt claim. The resulting receipt may support:

For reference `ref-cli-1.8.2`, fixture family X , and environment E , the candidate preserves malformed-input exit class for the minimized directive-error cases in court `cli-exit-minimize`.

It may not support:

The candidate is byte-identical on stderr, fully CLI-compatible, or a drop-in replacement for all malformed-input behavior.

This is the core developer habit FRF tries to install. A passing test is not the artifact. The artifact is the chain from admitted authority to raw capture, residual token, minimization, disposition, invariant, receipt, and bounded claim.

13 Court Cookbook

The framework becomes useful when courts are ordinary enough that developers can write them repeatedly. The following court types are practical starting points. They are examples, not an exhaustive taxonomy.

Court type	Question	Typical residuals
CLI court	Does the candidate preserve argument parsing, exit class, stdout/stderr routing, and diagnostics for a fixture family?	CLI, exit, text, ordering, platform.
Parser court	Does the candidate accept, reject, and classify source inputs the same way as the authority under a declared dialect or profile?	Semantic, text, harness, oracle-version, structured Unknown.
Wire court	Does the candidate emit or accept the same protocol bytes under a controlled peer exchange?	Wire, timing, ordering, security, state.
Filesystem court	Does execution create, remove, rename, chmod, truncate, flush, or materialize files in the same observable way?	Filesystem, permission, process, platform.
State-machine court	Does an interaction move through the same externally inferable states?	State, timing, ordering, recovery, authority conflict.
Persistence court	Does a stored artifact remain readable and equivalent for admitted downstream readers?	Filesystem, semantic, byte, version, downstream-reader.

Court type	Question	Typical residuals
Security-boundary court	Does the candidate preserve or intentionally strengthen an admitted boundary without claiming parity on changed behavior?	Security, permission, process, intentional divergence.
Performance-envelope court	Does a bounded resource claim hold under admitted inputs and host conditions?	Performance, platform, out-of-envelope, nondeterministic.
Freshness court	Are generated facts, ledgers, README claims, and receipts synchronized with machine-readable evidence?	Claim-boundary, harness, stale-documentation.
Vendor-oracle court	Is a candidate external executable or package admitted as an authority for this question?	Oracle-version, platform, non-admission, authority conflict.

The useful discipline is to keep each court narrow. A filesystem court should not silently license parser parity. A parser court should not license daemon replacement. A performance-envelope court should not license semantic correctness. Claims become credible when their surfaces remain separate.

14 Residual Taxonomy

Residual kind	Meaning
Wire	Difference in serialized bytes, canonicalization, compression, padding, checksums, reserved bits, or field ordering.
Semantic	Both sides produce valid outputs that mean different things.
State	Divergence in internal or externally inferable state transitions.
Timing	Differences in poll intervals, retries, expiry, scheduling, timeout boundaries, clock transitions, or temporal ordering.
Ordering	Permutations that may or may not be semantically significant.
Text	Diagnostics, formatted output, whitespace, and textual serialization.
CLI	Argument interpretation, option interaction, defaults, aliases, environment behavior, and invocation semantics.
Exit	Process result class or termination mode.
Filesystem	Filenames, directory structures, bytes, metadata, atomicity, durability, replacement policy, or cleanup.
Permission	Ownership, mode bits, credentials, capabilities, sandbox boundaries, or access decisions.
Process	Spawning, privilege separation, lifecycle, parent-child relationships, signals, or shutdown.
Log	Diagnostic or operational event streams.
Statistic	Counters, summaries, monitoring values, or aggregation behavior.
Control	Management protocols and administrative surfaces.
Security	Validation, trust boundary, authentication, cryptographic policy, or hardening behavior.

Residual kind	Meaning
Platform	Differences specific to operating systems, architectures, kernels, filesystems, libc implementations, or other environmental strata.
Performance	Throughput, latency, memory, CPU, energy, allocation, or scalability.
Nondeterministic	A measured difference demonstrated to arise from an intentionally variable quantity.
Oracle-version	Behavior that differs across admitted reference versions.
Harness	Difference caused by the measurement apparatus.
Unknown	A residual that remains unattributed.

The unknown category is essential. Without an explicit unknown state, pressure to complete a project tends to transform incomplete understanding into unsupported explanation.

15 Normalization as a Governed Operation

Normalization is necessary in real differential testing. Process identifiers vary. Temporary paths vary. Addresses may be allocated dynamically. Timestamps advance. Cryptographic nonces differ. Randomized hash iteration may reorder equivalent data. Absolute build paths differ.

The danger arises when normalization is treated as harmless preprocessing. FRF imposes four requirements:

1. Raw evidence precedes normalized evidence.
2. The normalization rule is versioned evidence.
3. The reason the field is considered non-semantic is documented.
4. The resulting claim does not include the property normalized away.

If two diagnostic lines differ only in absolute temporary directory, replacing the outer temporary directory with `<TMP>` may be admissible for a claim about diagnostic class and fixture-relative source location. It is not admissible for a claim of byte-identical stderr.

Principle. Every normalization spends evidence resolution. A growing normalization set is not automatically wrong, but it is a signal that the equivalence relation is becoming weaker.

16 Authority Separation

An oracle does not possess unlimited authority. A standards document may own normative protocol meaning. A historical binary may own compatibility behavior. A signed source archive may own release provenance. A distribution package may own evidence about a packaging ecology. A downstream reader may own evidence about consumer compatibility. A formal model may own a mathematically proven invariant. A benchmark environment may own a performance observation.

No witness should be promoted beyond its authority. If an RFC requires behavior *A* while an admitted historical implementation produces *B*, the reconstruction may reproduce *B* for drop-in compatibility or implement *A* for standards conformance. Either choice can be legitimate, but the authority conflict must remain visible.

17 Negative Capabilities

Most project documentation emphasizes what software can do. FRF additionally records what the evidence does not justify. A good receipt answers:

- What does this establish?
- What tempting stronger conclusion does it not establish?

Negative capabilities stop readers from inferring universal compatibility from finite tests, daemon replaceability from parser parity, cross-platform behavior from one host, or security from memory safety. Non-claims are not afterthoughts; they are engineering controls against claim inflation.

18 Developer Operating Rules

FRF is meant to change daily engineering behavior. The following rules are intentionally practical.

1. A green test without a bounded claim is a local signal, not an argument.
2. Compatibility is not a property of code alone; it is a property of code observed against an admitted authority under declared conditions.
3. The authority is not sacred. The observation is.
4. Preserve raw residuals before interpreting, normalizing, filtering, or fixing them.
5. Treat every normalizer as claim-weakening machinery until proven otherwise.
6. Never call an unmeasured surface compatible.
7. Never let a fixture count substitute for surface coverage.
8. Never delete a residual without a disposition.
9. Never convert an intentional divergence into a parity claim.
10. Never let the README outrun the receipts.
11. If a statement can be generated from evidence, generate it.
12. A receipt is the build artifact of a claim.

The last rule is the most important developer-design constraint. Claims should be built from source code, fixtures, admitted authorities, environments, comparators, raw captures, residual ledgers, and receipt schemas. A hand-written claim that cannot be traced into those artifacts is uncompiled prose.

Semantic non-bypass rule. No high-level verdict may bypass the evidence grammar. A benchmark score, detector count, green test, GPU kernel output, static-analysis score, or model-generated explanation cannot directly emit a compatibility claim. It must pass through the admitted authority, court envelope, raw capture, residual token, routed evidence, receipt, non-claims, and replay contract. This rule is what keeps accelerated or automated verification from becoming a faster way to overclaim.

19 Developer Design

A project does not need a large evidence platform to adopt FRF. The minimum useful form contains six objects: admitted authority, narrow court question, raw two-sided capture, residual ledger, receipt, and bounded claim. Everything else improves scale, rigor, or maintainability.

At larger scale, an FRF repository benefits from:

- an archaeology atlas indexing upstream APIs, source files, versions, protocols, and known behaviors;
- a court registry mapping surfaces to experiments;
- a residual store recording unresolved and historical mismatches;
- a residual corpus storing reusable residual fingerprints, dispositions, blind spots, and transfer boundaries;
- an invariant bank storing executable properties discovered from residuals and review findings;
- a receipt index mapping experiments to immutable evidence;
- a claim ledger mapping public statements to receipts;
- a negative-capability ledger recording forbidden conclusions;
- a freshness gate verifying generated documentation against current evidence;
- a version ladder replaying selected courts across historical references;
- an environment matrix exercising multiple execution ecologies;
- an independence map recording which witnesses derive from separate authorities.

Together these form an evidence graph:

$$\textit{Claim} \rightarrow \textit{Court} \rightarrow \textit{Receipt} \rightarrow \textit{Capture} \rightarrow \textit{Fixture} \rightarrow \textit{Authority}.$$

Side edges connect residuals, normalizers, witnesses, environments, versions, and invariants. A reviewer should be able to traverse from a claim to its evidence, from a residual to the claims it blocks, from an authority version to dependent receipts, and from a code change to courts requiring replay.

19.1 Invariant and Residual Corpora

FRF becomes more valuable when a resolved residual does not merely close a ticket. A residual that reveals a general failure class should be converted into reusable engineering knowledge.

An *invariant* states what must remain true under declared assumptions. A *behavioral fingerprint* names the minimal observables needed to evaluate that invariant without depending on incidental source structure. A *residual fingerprint* maps a divergence to a known failure family. An *invariant corpus* stores scoped invariants with provenance, applicability predicates, observables, negative controls, mutation operators, evidence tier, blind spots, and motivating residuals. A *residual corpus* stores raw and coarsened residuals, known dispositions, reproducer links, residual fingerprints, and unresolved structured Unknowns.

The practical loop is:

Observe $\rightarrow$ understand $\rightarrow$ fingerprint $\rightarrow$ generalize $\rightarrow$ encode invariant $\rightarrow$ encode residual fingerprint $\rightarrow$
 attack detector $\rightarrow$ preserve provenance.

The phrase *attack detector* is deliberate. A court should not be trusted merely because it detects the original defect. Where practical, each important invariant should have a negative control or mutation operator: an intentionally defective candidate that the court must reject. If the court cannot reject the seeded failure class, the invariant has not yet become operational evidence. This prevents circular verification, where the implementation, test, oracle wrapper, and explanation all share the same mistaken assumption.

Corpus discipline. Each invariant record should minimally include:

```
invariant:
  id: string
  claim: string
  applicability: string
  assumptions: [string]
  required_observables: [string]
  fingerprint_extractor: string
  violation_predicate: string
  negative_controls: [string]
  mutation_operators: [string]
  evidence_tier: string
  blind_spots: [string]
  motivated_by_residuals: [string]
```

This is not a replacement for human review. It is a way to stop humans from re-reviewing the same understood failure class forever. Human attention should move toward novel residuals, observability gaps, applicability failures, and adversarial cases that current courts cannot see.

19.2 Claim Compilation

Claim compilation is the process of generating human-readable compatibility statements from machine-readable evidence while rejecting statements that exceed receipt scope. It is the natural endpoint of FRF tooling.

authority + fixture + environment + capture + residuals → receipt → claim

A claim compiler should be conservative. If a receipt contains an open residual, the compiler should either block the positive claim or emit a weaker claim that names the residual. If a court used normalization, the compiled claim should avoid byte-identical language on the normalized surface. If only Linux was exercised, the claim should not say cross-platform. If only parser behavior was measured, the claim should not say daemon replacement.

This makes documentation less dependent on human restraint. The project can still contain explanatory prose, but the mechanically derivable part of trust should be mechanically derived.

Compiled claim example. Given a receipt with:

```
authority: tzcode-zic-2026b
court: canonical-zone-behaviour
fixtures: tzdata.zi canonical zones
observables: zdump output over 1900..2040
residuals: []
normalizers: []
platforms: [x86_64-linux]
```

a conservative claim compiler may emit:

For the admitted `tzcode-zic-2026b` authority, the candidate matched the `zdump` behavior oracle for the declared canonical-zone fixture set over the declared year horizon on the recorded Linux environment.

It must not emit:

The candidate is a complete `zic` replacement, byte-identical for all zones, cross-platform verified, or correct for all civil-time history.

The second sentence may contain ambitions, roadmap items, or separately measurable surfaces. It is not compiled from that receipt.

19.3 Repository Shape

A small implementation can use a simple layout:

```
frf/
  authorities/
    ref-cli-1.8.2.yaml
  courts/
    cli-diagnostic-minimize/
      manifest.yaml
      fixtures/
        run_authority.sh
        run_candidate.sh
        compare.sh
  captures/
  residuals/
  receipts/
  claims/
```

This layout is not normative. Its purpose is to show the separation of concerns: authorities are admitted once, courts ask narrow questions, captures preserve raw observation, residuals survive interpretation, receipts bind evidence, and claims are generated or checked against receipts.

20 Practical Methods from Public Oracle Repositories

The reconstruction repositories cited here show several recurring FRF methods in executable form. They are included as observational engineering examples, not as controlled evidence that the framework is universally superior.

This section is Tier 4 evidence: anecdotal from a single author's self-reported repositories, offered as implementation narrative and mechanism illustration rather than as validated results.

Self-authored materials are in scope for a conceptual prior-art document; they do not constitute independent replication or third-party validation of the case-study claims.

Method pattern	Practical use
Source archaeology atlas	<code>chrony-rs</code> and <code>ntpsec-rs</code> describe C source archaeology, directive extraction, option-table indexing, and generated parity matrices that turn source history into court questions rather than prose confidence [6, 9].

Method pattern	Practical use
Generated freshness gate	chrony-rs , openntpd-rs , and ntpsec-rs use cargo xtask gen or cargo xtask check style workflows so generated facts, README statements, parity matrices, and negative-capability ledgers cannot silently drift from code evidence [6, 8, 9].
Host-effect isolation	The time-daemon reconstructions keep the deterministic protocol or time-discipline brain separate from host clock, sockets, privileges, and OS mutation. This creates a courttable core while preserving narrow lab boundaries for real host effects [6, 8, 9].
Vendor-oracle admission	zic-rs distinguishes canonical upstream zic , vendor-packaged zic , glibc lineage behavior, distro packaging axes, and target images that ship data but no compiler. Non-admission is treated as evidence rather than a failed narrative [10].
Trust ladder	zic-rs organizes evidence as staged trust questions, close receipts, live status, non-claims, and open debts. This is the repository version of FRF’s claim ladder: each stage answers a different question and forbids collapse into a global verdict [10].
Byte-versus-behavior separation	zic-rs , gnucobol-rs , and ncurses-native separate byte identity, semantic behavior, diagnostics, platform, fixture horizon, and replacement readiness. This prevents a strong result on one axis from becoming an unearned claim on another [10, 11, 12].
Corpus admission	gnucobol-rs reports admitted COBOL units with validity profiles, oracle dialects, source formats, compiler options, copybooks, runtime configuration, and first-failure localization. This is FRF at corpus scale: the fixture is not just counted, it is admitted [11].
Negative-capability ledger	chrony-rs , zic-rs , and ncurses-native make non-claims visible next to capabilities: no production replacement claim, no unmeasured terminal claim, no universal zic claim, no byte-parity claim where only behavior was measured [6, 10, 12].
Operational parity matrix	zic-rs reports flag and mode rows as match, class parity, intentional divergence, unsupported-by-design, deferred, or divergent. This is a practical way to keep residual disposition separate from residual kind [10].
Receipt-bearing audit packet	zic-rs describes audit tools as witnesses whose folders are not automatically results; run status, receipts, proof scope, and non-green outcomes remain explicit. This prevents tooling presence from becoming evidence by implication [10].

These examples suggest a useful implementation rule: build the evidence system so the strongest impressive sentence cannot be written unless the receipts already license it. When a repository can generate a parity matrix, reject stale docs, name non-claims, route residuals, and admit or reject oracles by rule, FRF has moved from philosophy into daily engineering.

21 Flagship Case Files

The following case files show how FRF becomes practical engineering. They are not presented as universal proof of complete replacement for every environment. They are reusable patterns:

each case names an authority problem, a court shape, the residuals worth preserving, the receipt it can emit, and the claim it must block when evidence is missing.

This section is Tier 4 evidence: anecdotal from a single author’s self-reported repositories, offered as illustration of the mechanism rather than as validated results.

Every case study and most of the reference list is self-authored and in scope for a conceptual prior-art document; this does not constitute independent replication or third-party validation of the case-study claims.

Case	Central court	Residual language	Developer lesson
zic-rs	Time-zone compiler and reader behavior across admitted authorities.	Byte, behavior, vendor, reader, and horizon residuals.	Never collapse byte identity, downstream behavior, vendor behavior, and replacement readiness into one sentence [10].
chrony-rs	Time-discipline behavior separated from host clock and network mutation.	Host-effect, deterministic-core, directive, and freshness residuals.	Keep the brain courttable before admitting the machine that mutates the host [6].
gnucobol-rs	Corpus-scale COBOL behavior with independent parity maps.	Corpus-admission, diagnostic, runtime, AST, and call-edge residuals.	A corpus is evidence only after each unit is admitted with dialect, options, copybooks, and runtime context [11].
ntpsec-rs	Protocol, daemon, client, package-swap, and peer-interoperability courts.	Two-sided oracle, wire, migration, NTS-KE, configuration, and operational residuals.	A replacement claim needs courts for behavior, deployment mechanics, and neighbor systems, not only library tests [9].
m4-rs	Macro-language behavior split into named semantic axes.	Lexing, quote, argument, definition-stack, diversion, shell, frozen-file, hostile-input, and fuzz residuals.	Languages need court taxonomies; one green parser test does not cover a living semantics surface [7].
ncurses-native	Terminal byte-output parity for a bounded terminal and build surface.	Escape-sequence, cursor, SGR, color, frame, terminal-version, and unmeasured-surface residuals.	Low-level compatibility often lives in bytes; exact byte courts are powerful only inside an explicit terminal envelope [12].
openntpd-rs	Clean behavioral reconstruction with generated checks and no-original-code gates.	No-original-code, parity-before-label, host-boundary, generated-documentation, and operational residuals.	Do not let a project call itself ported, compatible, or clean until the gate that licenses that word has passed [8].

21.1 Case 1: zic-rs and Authority Ecology

Time-zone compilation has multiple plausible witnesses: upstream IANA **zic**, vendor-packaged **zic**, glibc-lineage behavior, distribution images, serialized TZif files, **zdump** readers, and target systems that may ship time-zone data without shipping a compiler. FRF treats these as separate authorities. A vendor executable may be admitted for that vendor package while not being admitted as a universal oracle for upstream **zic**. A target image that contains TZif data but no compiler may be evidence for deployed data consumption, not for compiler behavior.

Court pattern.

For admitted authority R , fixture family X , year horizon Y , downstream reader D , and environment E , does the candidate produce behavior-equivalent zone output under the declared comparison relation?

The court captures compiler inputs, authority identity, candidate identity, generated artifacts, downstream reader output, exit status, diagnostics, and environment digest. Raw generated bytes and downstream behavioral observations are both preserved, because byte identity and behavior identity are different claims.

Residual tokens.

```
byte/tzif/layout-change/open -> court: tzif-byte-minimize
semantic/zdump/horizon-match/pass -> receipt: behavior-horizon
oracle/vendor-conflict/non-admitted -> court: authority-review
platform/reader-only/evidence-boundary -> non_claim: compiler-parity
```

A byte residual does not automatically mean behavioral failure. A behavioral match does not automatically license byte identity. A vendor mismatch does not automatically make either side wrong; it may expose an authority boundary. This is the practical shape of residual primacy: the remainder survives long enough to become a route.

21.2 Case 2: chrony-rs and Host-Effect Isolation

Time software is difficult because the interesting algorithm is surrounded by host effects: clocks, sockets, privileges, leap handling, kernel discipline, service management, peer state, and wall-clock nondeterminism. The **chrony-rs** pattern separates a deterministic core from host mutation so the discipline logic can be courted before a deployment claim is attempted.

Court pattern.

For admitted chrony authority R , directive set D , synthetic observation stream S , and deterministic environment E , does the candidate produce the same state-transition and decision trace under the declared comparison relation?

The court should capture directive parsing, normalized configuration, input observations, decision trace, state transitions, emitted actions, and a record of effects intentionally not exercised. Host effects become named boundaries rather than hidden assumptions.

Residual tokens.

```
directive/parse/table-drift -> court: directive-admission
core/state-transition/open -> court: discipline-trace-minimize
host/clock-mutation/not-exercised -> non_claim: production-daemon
docs/generated-stale/fail -> gate: freshness
```

The lesson is not to avoid host behavior. The lesson is to admit it in layers. A deterministic core court can support a bounded algorithmic claim. It cannot support a production daemon claim unless the host-effect courts also emit receipts.

21.3 Case 3: gnucobol-rs and Corpus Admission

Large language implementations tempt developers to count tests as if every fixture means the same thing. COBOL makes that unsafe: dialect, source format, copybooks, runtime configuration, diagnostics, generated code, data representation, and call edges all change what a fixture means. The **gnucobol-rs** pattern treats a corpus as an admitted population, not a bag of files.

Court pattern.

For admitted COBOL unit U , dialect D , source format F , compiler options O , copybook closure C , and runtime configuration E , does the candidate match the declared compile, diagnostic, runtime, and structural observables?

Independent parity maps matter here. A compile-output court, a runtime court, a Doxygen-style surface map, and a Clang-AST or call-edge map do not answer the same question. Agreement across maps is stronger than one aggregate pass count, but each map still licenses only its own claim.

Residual tokens.

```
corpus/unit/not-admitted -> court: fixture-admission
diagnostic/message-order/open -> court: diagnostic-parity
runtime/data-layout/open -> court: runtime-observable
map/call-edge/missing -> court: structural-index
```

The practical rule is simple: no corpus claim without corpus admission. The receipt should name which units were admitted, which were rejected, why they were rejected, and which failure family occurred first.

21.4 Case 4: ntpsec-rs and Migration-Grade Parity

Protocol replacements need more than function-level parity. A daemon and client suite must satisfy wire behavior, control commands, configuration behavior, service behavior, package mechanics, and peer interoperability. The `ntpsec-rs` pattern is useful because it treats replacement readiness as a ladder of courts rather than a single final assertion.

Court pattern.

For admitted NTPsec authority R , fixture family X , peer or client P , environment E , and operational mode M , does the candidate match the declared observable behavior and preserve the migration boundary?

Two-sided oracle courts are especially valuable in this class. A candidate can be compared against the reference, then exercised against an independent peer such as another time implementation. The second court does not prove universal interoperability, but it can expose residuals that same-oracle testing misses.

Residual tokens.

```
wire/packet-byte/open -> court: packet-minimize
control/command-diagnostic/open -> court: client-suite-parity
interop/nts-ke/peer-drift -> court: peer-interoperability
package/swap/unproven -> non_claim: drop-in-replacement
```

The developer lesson is that migration itself is a surface. A library may pass behavioral courts and still fail the package-swap court. FRF makes that failure visible before users discover it operationally.

21.5 Case 5: m4-rs and Language-Surface Courts

Macro languages hide compatibility in small textual details: quote nesting, token boundaries, argument collection, definition stacks, diversions, diagnostics, include search, shell execution, frozen files, and malformed input. The `m4-rs` pattern names those surfaces as separate courts instead of treating the language as a monolith.

Court pattern.

For admitted GNU m4 authority R , semantic axis A , fixture family X , and sandbox policy S , does the candidate match byte output, diagnostics, exit behavior, and side-effect boundary under the declared comparison relation?

The axis list is the practical artifact. A maintainer can see whether a residual belongs to lexing, quoting, argument expansion, `pushdef`, diversion, diagnostics, shell builtins, frozen files, hostile input, or fuzz replay. That coarsening gives the language a residual grammar.

Residual tokens.

```
lex/byte-tokenization/open -> court: lexer-minimize
quote/nesting/divergent -> court: quote-axis
shell/syscmd/sandbox-boundary -> court: side-effect-admission
hostile/input/no-panic-pass -> receipt: robustness-axis
```

The useful discipline is that language compatibility advances axis by axis. A parser win does not erase a diagnostic residual. A no-panic result does not imply byte parity. Each court keeps its own receipt.

21.6 Case 6: ncurses-native and Byte-Exact Terminal Envelopes

Terminal compatibility is a hard reminder that behavior can be bytes. Cursor motion, SGR sequences, color setup, screen framing, and terminal-version assumptions are not incidental when downstream programs consume the byte stream. The `ncurses-native` pattern is strongest when it states its terminal envelope plainly.

Court pattern.

For admitted ncurses build R , terminal description T , fixture screen X , and environment E , does the candidate emit the same terminal byte stream under the declared comparison relation?

Byte courts are strict, but they must not become global. A byte-exact result for `xterm` under an ncurses build does not establish every terminfo entry, input behavior, panels, windows, or unrelated terminal families.

Residual tokens.

```
byte/escape-sequence/open -> court: terminal-byte-minimize
cursor/optimization/different -> court: cursor-path
terminfo/unmeasured -> non_claim: all-terminals
surface/input/not-covered -> non_claim: full-curses
```

The developer lesson is to treat exactness and scope as a pair. Exact bytes are excellent evidence inside an envelope and poor evidence outside it.

21.7 Case 7: openntpd-rs and Parity-Before-Label Gates

The `openntpd-rs` pattern is valuable because it makes naming discipline executable. A repository should not casually claim “ported”, “compatible”, “clean”, or “drop-in” as branding. Those words should be claim outputs, gated by receipts.

Court pattern.

For admitted OpenNTPD authority R , source-archaeology record A , generated evidence set G , and behavior court family C , does the candidate preserve the declared behavior without relying on original implementation code?

The no-original-code gate is not a behavioral court; it is a design and process court. The generated-documentation gate is not a protocol court; it is an anti-drift court. The operational courts are separate again. Keeping these gates separate prevents a clean process result from being mistaken for network parity, and prevents a network result from being mistaken for a process claim.

Residual tokens.

```
process/original-code/dependency-found -> block: clean-reconstruction
docs/generated-drift -> gate: xtask-check
behavior/parity-open -> block: compatible-label
host/mutation-boundary-open -> non_claim: production-replacement
```

This is the case-file rule developers should copy: important public words should be generated only after the repository can name the court that licenses them. If the court is absent, the word remains a roadmap item, not a claim.

21.8 Reusable Flagship Pattern

Across these cases, the strongest reusable pattern is:

```
authority -> claim -> court -> raw observation -> residual
          -> residual token -> route -> negative control
          -> receipt -> claim/non-claim compiler
```

The pattern is deliberately mechanical. It gives future projects a way to turn disagreement into work queues, design gates, court inventories, and claim boundaries. It also keeps the framework honest: the repository may make a strong claim only where the receipts support it, and it can still make practical progress while many residuals remain open.

22 The Taste Codex as Executable Developer Judgment

FRF tooling should embody the same engineering taste it asks from the software under test. The Taste Codex layer is operational: implementation disciplines, failure patterns, review rules, and design constraints are converted into artifacts that a repository can execute, attack, replay, and preserve.

The core translation is:

$$\begin{aligned} \text{Taste rule} &\rightarrow \text{Invariant} \rightarrow \text{Court} \rightarrow \text{Residual fingerprint} \\ &\rightarrow \text{Negative control} \rightarrow \text{Receipt} \rightarrow \text{Claim blocker.} \end{aligned}$$

This matters because many serious software failures are not syntax failures. They are failures of judgment: the wrong representation is chosen early, invalid states remain representable, a safety boundary leaks into callers, an API permits misuse, a performance claim lacks a disconfirming benchmark, or a compatibility repair is treated like greenfield design. FRF treats those failures as courttable surfaces.

The practical value is direct: a developer should be able to encode a design rule once, attach a falsifier, seed a negative control, and prevent future claims from bypassing that rule. The Taste Codex gives FRF a stable way to ask whether a rule is enforced, whether the detector can be fooled, and which claims must be blocked when the rule fails.

22.1 What the Taste Layer Adds

Conventional tests usually ask whether the candidate returns the expected output. The Taste Codex layer asks whether the implementation shape preserves the engineering constraints that make future behavior understandable, auditable, and difficult to misuse. A candidate may match an oracle on a fixture and still choose a representation that admits impossible states, expose a boundary that every caller must reason about, or publish an API whose common wrong use is shorter than correct use. FRF should not silently treat those as untestable opinions.

The Taste Codex extends FRF from behavioral parity to engineering-judgment parity. It does not replace output courts. It adds design courts that produce residuals when the implementation violates a declared design constraint. Those residuals may not prove a user-visible defect today, but they can block claims such as constant-time operation, contained boundary, misuse-resistant API, maintainable representation, compatibility-preserving migration, or measured performance envelope.

Taste constraint	FRF consequence
Invariant-first design	Write the invariant ledger before writing the court. A court that lacks a falsifiable invariant usually becomes a script that produces comforting output.
Representation before branching	If a court has many special cases, ask whether the residual representation is wrong. A good token grammar should make common residual paths uniform.
Unsafe or authority quarantine	Keep host mutation, privileges, raw pointers, shell effects, clocks, and external authorities behind small admitted boundaries. The dangerous region should be smaller than the concept it implements.
Compatibility repair mode	When changing behavior, classify the work as greenfield, migration, or compatibility repair. Installed behavior is part of the problem statement, not an inconvenience to edit out.
Simplicity with accounting	Do not call an evidence platform simple until state, ownership, time, failure, dependency surface, and local auditability have been counted.
Performance grounding	No performance or scale claim should appear without a named hot path, workload, measurement, and benchmark that could disprove the claim.
Misuse-resistant APIs	The correct way to add a court, normalizer, or claim should be shorter than the incorrect way. APIs should make claim inflation hard to express.
Tooling that teaches	Repeated review findings should become checks, diagnostics, fix-it hints, negative controls, and regression courts. Practical judgment should become executable repository knowledge.

These constraints matter because verification tooling can itself become a source of false confidence. FRF courts should be designed so their own failure modes are inspectable: what they observe, what they normalize, what they cannot see, what assumptions they share with the candidate, and what seeded defects they fail to catch.

22.2 Repository Shape

A practical repository can make the Taste Codex layer concrete without a large platform:

taste/

```

gates/
  representation.yaml
  invariant-ledger.yaml
  boundary-quarantine.yaml
  api-misuse.yaml
  compatibility-repair.yaml
  performance-envelope.yaml
  simplicity-accounting.yaml
courts/
  representation-edge-case/
  invalid-state/
  boundary-quarantine/
  api-misuse/
  compatibility-replay/
  benchmark-envelope/
  local-auditability/
negative_controls/
  vec-remove-zero-queue/
  bool-parameter-api/
  leaky-ffi-wrapper/
  greenfield-breaking-change/
residual_fingerprints/
claim_blockers/

```

The layout is intentionally ordinary. The important point is that taste gates live beside courts, negative controls, residual fingerprints, and claim blockers. The design rule is not a paragraph in a guide. It is a replayable object that can fail.

22.3 Gate Record

Each gate should state the design constraint, the surface it observes, how it is falsified, which negative controls must fail, and which claims are blocked when it emits a residual:

```

taste_gate:
  id: representation.queue.front_removal
  constraint: front removal must not shift all remaining elements
  applies_to:
    surfaces: [queue, ring_buffer, deque, scheduler_ready_list]
    claims: [bounded_queue_operation, predictable_latency]
  observables:
    - operation_count
    - allocation_count
    - elapsed_envelope
    - implementation_path
  falsifier: repeated front removal exhibits linear element movement
  negative_controls:
    - vec_remove_zero_queue
  residual_fingerprints:
    - queue_linear_front_removal
  claim_blockers:
    - no O(1) pop-front claim
    - no predictable-latency claim under this workload

```


This schema is deliberately modest. A gate does not need to decide everything about a design. It needs to make one important judgment executable and bounded.

22.4 Taste-to-FRF Translation

Taste signal	FRF artifact	Court example	Residual if violated
Invariant-first design	Invariant ledger	State/property court checks whether declared impossible states remain impossible.	Missing-invariant residual; undocumented acceptance boundary.
Representation before branching	Representation invariant and minimizer	Edge-case minimization court checks whether a special path is a domain fact or representation artifact.	Edge-case residual; duplicated-path residual; redundant-state residual.
Unsafe boundary discipline	Boundary admission and safety court	FFI, privilege, raw-pointer, clock, shell, or host-mutation court keeps dangerous authority behind a narrow wrapper.	Boundary-leak residual; caller-obligation residual; unquarantined-authority residual.
API misuse resistance	API surface court	Invalid-call construction court checks whether wrong use is easier to express than correct use.	Misuse-permitted residual; boolean-parameter residual; hidden-ownership residual.
Compatibility repair mode	Migration or repair court	Old/new behavior replay court checks that compatibility claims survive declared migration mechanics.	Claim-drift residual; unsupported-breaking-change residual.
Simplicity accounting	Auditability court	Dependency/state/failure review court checks whether the design can be locally audited under partial failure.	Hidden-state residual; dependency-surface residual; unauditability-abstraction residual.
Performance grounding	Benchmark-envelope court	Hot-path replay court requires workload, measurement, and a benchmark that could disprove the speed claim.	Unsupported-performance-claim residual; workload-out-of-envelope residual.
Tooling that teaches	Regression court and freshness gate	Repeated-failure court converts a review finding into a reusable detector, negative control, and receipt.	Repeated-judgment residual; stale-guidance residual.

22.5 Design Gates

The Taste Codex contributes concrete gates that can be placed before implementation or review. FRF uses these gates to prevent taste from remaining implicit.

Invariant ledger.

Before building a court or feature, state the invariants, where they are enforced, how they can be violated, and which test would falsify them. An unenforced invariant is a future residual. The gate blocks claims that the design is constrained, safe by construction, or locally auditable.

Representation check.

Before adding a branch for first, last, empty, null, head, sentinel, or special cleanup behavior, ask whether a different representation makes the normal path include the case. If the special case is not a domain fact, it should be treated as a representation residual. The gate blocks claims about simple representation, reviewability, and predictable behavior where hidden cases remain.

Invalid-state gate.

List states the representation permits but the domain forbids. Each forbidden state must be impossible by type, impossible by constructor, rejected at runtime, or explicitly recorded as caller obligation. The gate emits an invalid-state residual when the code admits a forbidden state without an enforcement site.

Boundary quarantine.

Unsafe code, host mutation, privileges, raw pointers, shell effects, clocks, external commands, and authority witnesses should be narrower than the concept they implement. A wrapper that exports its danger to every caller has not contained the boundary. The gate blocks contained-boundary and safe-wrapper claims.

Compatibility classification.

Any change to observable behavior must be classified as greenfield, migration, or compatibility repair. A greenfield redesign is not evidence against installed reality. The gate blocks compatibility-preserving claims until old behavior, new behavior, breakage, migration path, rollback, and replay courts are present.

Simplicity accounting.

Before accepting an abstraction, count state, ownership, time, failure behavior, dependency surface, and local auditability. “Simple” is a claim requiring evidence. The gate emits hidden-state, temporal-coupling, dependency-surface, or unauditable-abstraction residuals.

Performance grounding.

A performance claim must name the hot path, data layout, allocation behavior, workload, and benchmark that could disprove it. Without a disconfirming benchmark, the claim is not yet admissible. The gate blocks faster, scalable, constant-time, bounded-latency, and low-overhead claims until receipts exist.

Misuse resistance.

Correct use should be shorter than incorrect use. If a public API makes wrong behavior easy to express, FRF should treat that as an observable design residual, not a matter of taste alone. The gate blocks misuse-resistant and safe-default claims.

Tooling that teaches.

A repeated review finding should become a detector, a fix-it hint where possible, a negative control, and a regression court. The gate emits a repeated-judgment residual when a known failure class reappears without an executable guard.

22.6 Worked Design Courts

The following examples show how the Taste Codex layer becomes FRF machinery.

Representation court: front removal. Consider a queue-like component implemented with a growable array and front removal that shifts every element. A shallow review says “use a better data structure.” FRF records a stronger artifact chain:

```
taste_rule: representation before branching
invariant: pop_front must not shift all remaining elements
fingerprint:
  observables: operation_count, allocation_count, elapsed_envelope
negative_control:
  implementation: vec_remove_zero_queue
court:
  fixture: repeated_push_pop_workload
  falsifier: linear element-shift behavior under declared workload
residual_fingerprint:
  queue-linear-pop-performance-residual
claim_blocker:
  no O(1)-queue-operation claim under this representation
```

The point is not that every queue requires benchmarking ceremony. The point is that an important review judgment can become reusable. Once a failure class is understood, future systems should not require a human to rediscover the same reasoning from scratch.

API misuse court: boolean parameter surface. An API that accepts several boolean parameters can make incorrect use compact and correct use unreadable:

```
gate: api_misuse.boolean_parameters
constraint: public API must name independent variation points
negative_control:
  function: process(path, true, false, true, false)
court:
  checks:
    - call_site_readability
    - invalid_combination_representability
    - extensibility_without_boolean_growth
residual_fingerprint: boolean_parameter_misuse_surface
claim_blocker:
  - no misuse-resistant API claim
  - no self-documenting call-site claim
```

The repair is not automatically a specific pattern. It may be an options struct, typed enum, builder with validation, typestate transition, or narrower function split. The court only says the existing API permits misuse too easily under the declared gate.

Boundary quarantine court: unsafe or host authority. A boundary wrapper should prevent callers from reasoning about raw handles, shell effects, host clocks, privileges, or external authority mechanics:

```
gate: boundary_quarantine.ffi_or_host_effect
constraint: dangerous authority remains behind narrow admitted boundary
observables:
  - public_api_signature
  - raw_handle_exposure
  - caller_obligations
```

```

    - effect_surface
negative_control:
    wrapper_exposes_raw_pointer_or_shell_effect
residual_fingerprint: boundary_leak
claim_blocker:
    - no contained-boundary claim
    - no safe-wrapper claim

```

This court is especially important for FRF itself. A parity runner that mutates host clocks, shells out to unpinned tools, or exposes authority selection to arbitrary callers can corrupt its own evidence.

Compatibility repair court: installed behavior. A behavior change must not be presented as a compatibility-preserving improvement unless the migration evidence exists:

```

gate: compatibility_repair.observable_change
classification: greenfield | migration | compatibility_repair
required_for_repair:
    old_behavior: receipt
    new_behavior: receipt
    breakage_surface: ledger
    migration_path: receipt
    rollback_plan: receipt
    replay_courts: [old, new, migrated]
negative_control:
    greenfield_redesign_without_migration
residual_fingerprint: compatibility_claim_drift
claim_blocker:
    - no compatibility-preserving change claim

```

This gate keeps elegant redesign from overwriting installed reality. If the project chooses a breaking improvement, FRF can record that honestly. What it cannot do is call the break parity.

Simplicity court: local auditability. Simplicity is often asserted when the design merely has fewer lines. FRF requires an accounting surface:

```

gate: simplicity.local_auditability
observables:
    - state_count
    - ownership_paths
    - temporal_coupling
    - partial_failure_paths
    - dependency_surface
    - local_replay_command
residual_fingerprint:
    hidden_state | temporal_coupling | dependency_surface |
    unauditably_abstraction
claim_blocker:
    - no simple-design claim
    - no locally-auditably claim

```

The court does not punish abstraction. It asks whether the abstraction reduces the reasoning burden under failure, time, ownership, and dependency pressure.

Performance court: disconfirming benchmark. Performance claims should enter FRF only with a workload and a way to be wrong:

```
gate: performance.disconfirming_benchmark
required:
  hot_path: string
  data_layout: string
  allocation_per_operation: string
  workload: string
  benchmark_that_disproves_claim: command
residual_fingerprint:
  unsupported_performance_claim | out_of_envelope_workload
claim_blocker:
  - no faster claim
  - no bounded-latency claim
  - no constant-time claim
```

This gate is not about making every project performance-heavy. It prevents performance language from outrunning measurement.

23 Adoption Path

FRF should not require a large platform before it produces value. A practical adoption ladder is:

Level 0.

Run authority and candidate manually; save raw outputs.

Level 1.

Add a fixture, comparator, and residual file.

Level 2.

Add authority admission and a receipt.

Level 3.

Generate claim and non-claim text from receipts.

Level 4.

Add CI replay and anti-drift gates.

Level 5.

Add version, environment, and independent-witness matrices.

The correct level depends on risk. A one-off internal migration may only need Levels 0–2. A public compatibility claim for infrastructure should move toward Levels 3–5. The point is not ceremony. The point is to make unsupported confidence progressively harder.

24 Empirical Evaluation Design

This paper presents FRF primarily as a methodology derived from reconstruction practice. A stronger empirical case requires controlled evaluation. The following research questions are directly testable:

- RQ1.** Does residual primacy reveal compatibility defects that conventional candidate-only testing misses?
- RQ2.** Does FRF reduce claim inflation when independent reviewers compare public claims against available evidence?
- RQ3.** How much does minimization reduce residual-resolution cost?
- RQ4.** How stable are receipts across commits, toolchain changes, host kernels, and reference upgrades?
- RQ5.** Which residual classes are most productive in defect discovery?
- RQ6.** Does source archaeology improve edge-case discovery compared with unguided generation?
- RQ7.** What engineering overhead does FRF impose in CI time, storage, tooling, and developer effort?
- RQ8.** Does explicit negative capability improve reviewer calibration?
- RQ9.** How transferable is FRF across utilities, compilers, databases, protocol daemons, file-format tools, GUI systems, and libraries?
- RQ10.**
Do invariant-derived residual fingerprints detect semantically equivalent failure classes across independently structured implementations?
- RQ11.**
Do negative controls and mutation operators reduce false confidence in courts and invariant banks?
- RQ12.**
Which observability gaps most often allow seeded defects to escape court detection?

The relevant success metric is not simply more tests. A stronger evaluation measures reduction of unexplained behavioral surface per unit of engineering effort.

25 Quantitative Metrics

Metric	Interpretation
Claim-to-receipt ratio	Fraction of externally visible compatibility claims with direct receipt support.
Open residual density	Unresolved residuals relative to measured surfaces.
Normalization ratio	Fraction of observed fields requiring normalization; a proxy for evidence-resolution loss.
Replay success rate	Whether historical receipts remain executable.
Unknown-surface count	Target behavior without behavioral evidence.
Residual closure latency	Time from first observation to disposition.
Minimization factor	Original reproducer complexity versus minimized reproducer complexity.
Independent-witness count	Distinct evidentiary lineages for important claims.
Version span	Historical reference releases exercised.
Environment span	Distinct execution ecologies measured.

Metric	Interpretation
Claim-boundary violations	Incidents where documentation stated more than receipts supported.
Review-surface compression ratio	Raw differences or alerts divided by typed residual episodes; measures developer review load reduction while preserving raw evidence links.
Structured-unknown rate	Fraction of residual episodes with nontrivial residual signs but no admitted explanation; useful for detecting where the heuristics bank or court set needs expansion.
Out-of-envelope rate	Fraction of attempted executions that fall outside declared admissibility envelopes; prevents silent conversion of invalid measurements into passes.
Mutation kill rate by invariant	Fraction of seeded defects rejected by the courts that claim to enforce a given invariant.
Residual fingerprint transfer rate	Fraction of known residual fingerprints that detect semantically equivalent defects in independently structured implementations.
False confidence incident rate	Cases where a court, invariant, generated claim, or receipt reports confidence later contradicted by a residual or reviewer finding.
Observability escape rate	Fraction of seeded defects missed because the court did not observe the relevant surface.

These metrics require interpretation. Discovering more residuals can temporarily worsen residual density while representing progress. A lower normalization ratio is not automatically better if the domain contains legitimate nondeterminism. A high review-surface compression ratio is useful only if raw evidence remains reachable. Metrics are instruments, not replacements for engineering judgment.

26 Limits of Equivalence Claims

No practical FRF campaign over a large general-purpose program can usually prove universal equivalence. The input space may be infinite. The environment space may be effectively unbounded. Concurrency multiplies possible traces. Operating-system interfaces change. Undefined or implementation-defined behavior complicates authority. Hardware faults occur. Dependencies evolve. External networks introduce independent state.

FRF responds by refusing universal language. Instead of “the reconstruction is identical to the reference,” a bounded claim should read:

For reference release R , fixture family X , environment family E , and observable set Ω , the candidate produced no unexplained residuals under courts $C_1, \dots, C_n$.

This sentence is longer because it encodes the boundary of knowledge.

27 Developer Failure Modes

FRF is designed around ordinary developer failure modes. The following failures are not moral defects; they are predictable ways evidence degrades under schedule pressure.

Failure mode	FRF response
The candidate is written first and tests encode its behavior.	Run oracle experiments before implementation when possible; treat candidate behavior as a hypothesis, not an authority.
The installed reference changes under the test suite.	Require authority admission with version, path, hash, and environment.
The harness normalizes away the defect.	Preserve raw capture; version normalizers; weaken claims on normalized surfaces.
Parser parity is reported as application parity.	Compile claims only from court scope; emit non-claims for unmeasured surfaces.
Output bytes match but filesystem side effects differ.	Declare observable axes explicitly; add filesystem courts where side effects matter.
The harness is treated as neutral.	Test comparators and runners adversarially; include harness identity in receipts.
A fixed residual disappears from project memory.	Convert the minimized reproducer into a regression invariant.
The README says more than the receipts.	Generate or check claims from receipt scope.
An intentional safety change is described as compatibility.	Record intentional divergence and block parity language on that surface.
One host is treated as all hosts.	Separate environment span from universal claims.
Unknown behavior is silently counted as pass.	Maintain UNKNOWN, BLOCKED, and RESIDUALS-OPEN states distinct from PASS.

28 Threats to Validity

The method described here originated through a family of reconstruction projects with overlapping design goals and, in several cases, overlapping implementation language. Some practices may reflect those projects rather than universally optimal software-engineering practice. The case-study material is observational rather than controlled experimental evidence. It demonstrates that the framework can be instantiated, not that it is superior to every alternative workflow.

Specific repository states, court counts, and implementation details evolve. They should be treated as dated snapshots rather than permanent characteristics. The terminology of courts, receipts, forensic evidence, and evidence closure is organizational language; it should not be confused with legal evidence standards, cryptographic attestation, or mathematical proof.

FRF can also become excessively expensive if applied indiscriminately. Not every application requires byte-exact historical compatibility. For new standalone systems without a meaningful external authority, conventional specification-driven development may be more appropriate. The framework is strongest where a claim depends on an admitted authority: a mature reference implementation, public contract, protocol, persistence format, safety boundary, migration promise, prior release, benchmark envelope, policy, or independent oracle.

29 Discussion

FRF treats trust as an artifact graph rather than an adjective. Software projects often describe themselves as trustworthy, secure, compatible, production-ready, or drop-in. FRF asks what objects make those statements auditable.

Negative knowledge is valuable. Knowing that behavior has not been measured is superior to a misleading green status. Knowing that a residual is unexplained is superior to an invented rationale.

Residuals concentrate knowledge. The obvious behavior of a system is often easy to reproduce. The remaining disagreements disproportionately encode quirks, historical decisions, boundary conditions, and undocumented assumptions. This is why residual primacy has value beyond testing.

Software archaeology and executable testing reinforce each other. Archaeology finds questions. Courts answer them. Residuals reveal where archaeology was incomplete. Explanation sends the engineer back into history. This feedback loop is especially relevant when AI-assisted implementation increases code-production velocity: trust must move away from authorship and toward reproducible observations.

30 Conclusion

The hard problem in trustworthy software is not producing code. It is determining exactly which behavior has been observed, which authority licenses that behavior, which residuals remain, and where each claim stops being true.

The Forensic Residual Framework addresses this problem by placing disagreement at the center of development and reconstruction. An authority is admitted rather than assumed. Behavior is observed before it is claimed. Raw evidence is captured before it is normalized. Differences become residuals before they become explanations. Deterministic endoduction coarsens those residuals into a typed language that the codebase can route into courts, minimizers, invariants, and claim blockers. Defects become regression invariants. Intentional differences become explicit divergences. Unknowns remain unknown. Courts turn broad behavioral ambitions into narrow falsifiable questions. Receipts bind those questions to reproducible evidence. Claim ladders prevent one form of evidence from silently becoming another.

The framework's central rule is: do not optimize away the remainder before understanding what the remainder is telling you. The goal is not the largest possible number of passing tests or the strongest possible compatibility language. The goal is to steadily reduce unexplained behavior while preserving exact provenance for every conclusion reached.

In FRF, trustworthy software claims are not merely written. They are compiled from admitted authorities, raw observations, preserved residuals, deterministic residual tokens, dispositions, receipts, and explicit non-claims.

A Minimal Receipt Schema

```

receipt:
  schema_version: string
  court:
    id: string
    question: string
    falsifier: string
    admissibility_envelope:
      authority_versions: [string]
      fixture_family: string
      platforms: [string]
      observables: [string]
      normalizers: [string]
      replay_scope: string
  authority:
    name: string
    kind: executable_reference | specification | contract |
         schema | model | prior_release | policy
    version: string
    identity_hash: string
    provenance: string
  candidate:
    name: string
    version_or_commit: string
    build_profile: string
  environment:
    os: string
    architecture: string
    toolchain: string
    environment_digest: string
  fixtures:
    - id: string
      hash: string
      arguments: [string]
  observables:
    - axis: string
      raw_reference_hash: string
      raw_candidate_hash: string
      comparator: string
      normalization_rules: [string]
      verdict: pass | residual | blocked
  residuals:
    - id: string
      kind: string
      sign:
        norm: string
        drift: string
        slew: string
      grammar_state: admissible | boundary | violation |
                     recovery | unknown | intentional_divergence

```

```
raw_reference_hash: string
raw_candidate_hash: string
disposition: open | fixed | intentional | environmental |
              oracle_version | harness | nondeterministic | unknown
reproducer: string
invariant: string
residual_fingerprint: string
endoduction:
  schema_version: string
  tokens:
    - residual_id: string
      token: string
      next_court: string
      blocks_claims: [string]
  invariants:
    - id: string
      claim: string
      applicability: string
      fingerprint_extractor: string
      violation_predicate: string
      negative_controls: [string]
      mutation_operators: [string]
      blind_spots: [string]
  taste_gates:
    - id: string
      constraint: string
      gate_type: invariant_ledger | representation |
                  invalid_state | boundary_quarantine |
                  compatibility_repair | simplicity |
                  performance | misuse_resistance |
                  tooling_that_teaches
      observables: [string]
      falsifier: string
      negative_controls: [string]
      residual_fingerprints: [string]
      claim_blockers: [string]
  claims:
    positive: [string]
    non_claims: [string]
    blocked_by_open_residuals: [string]
  verdict_case_file:
    authority_hash: string
    fixture_hash: string
    environment_hash: string
    comparator_hash: string
    residual_token_hash: string
    stage_hashes: [string]
    final_case_hash: string
  replay:
    command: string
```

B Developer Checklist

1. Admit the authority before citing it.
2. State the court question narrowly.
3. Declare the admissibility envelope before running the court.
4. Define falsification conditions before comparison.
5. Capture raw authority and candidate observations before normalization.
6. Persist every mismatch as a residual before assigning cause.
7. Record residual sign fields: magnitude, drift or persistence, and slew or abruptness.
8. Coarsen residuals into deterministic evidence tokens without deleting raw evidence.
9. Preserve structured Unknown rather than collapsing it into failure or pass.
10. Separate residual kind from residual disposition.
11. Use residual tokens to route minimization, follow-up courts, and claim blocking.
12. Minimize residual reproducers when practical.
13. Convert resolved residuals into regression invariants.
14. For important invariants, add a negative control or mutation operator that the court must reject.
15. Encode relevant Taste Codex gates as executable constraints, not prose guidance.
16. For each design claim, name the gate, falsifier, negative control, residual fingerprint, and claim blocker.
17. Do not emit simple, safe-wrapper, misuse-resistant, compatibility-preserving, or performance claims unless the corresponding gate receipts exist.
18. Track residual fingerprints only with applicability predicates and known blind spots.
19. Treat observability as reviewable: record what the court cannot see.
20. Record intentional divergences as divergences, not parity.
21. Enforce the semantic non-bypass rule: no claim emitted directly from a score, detector, or green test.
22. State non-claims next to positive claims.
23. Keep generated facts generated.
24. Track review-surface compression only when raw evidence remains reachable.
25. Re-run receipts after code, oracle, toolchain, or environment changes.

C Claude Skill Appendix: FRF Quick Reference

This appendix converts `frf-quick-reference.md` into a compact Claude-skill-shaped operating guide. The paper remains the source of truth for the full framework, DSFB lineage, Taste Codex layer, formal treatment, scope limits, and evidentiary qualifications. This appendix exists so a developer or assistant can apply the operating core without first reading the full paper.

C.1 Suggested Skill Header

```
---
name: frf-quick-reference
description: Apply the Forensic Residual Framework (FRF)
            to software compatibility, reconstruction, verification,
            residual handling, court design, evidence receipts, and
            claim/non-claim compilation.
---
```

C.2 Part 1: Rosetta Table

If a reader already knows adjacent software practices, Table 11 gives the closest mapping and the added FRF discipline.

Table 11: FRF Rosetta Table

Adjacent engineering practices and the specific discipline FRF adds.

FRF term	Closest known practice	What FRF adds
Authority	Test oracle; source of truth in golden-master testing.	Requires admission: a versioned, hashed identity record before the oracle can be cited, preventing silent oracle drift.
Admission	Pinning a dependency version or lockfile.	Applied to evidence sources, not only build dependencies; the reference binary itself gets a lockfile-style entry.
Court	A single contract test or differential-testing harness run.	Must declare its admissibility envelope up front; a court outside declared conditions returns Unknown, not Pass.
Admissibility envelope	Test scope or preconditions in a test plan.	Made a first-class checked object rather than a comment or doc string.
Observable	Assertion target or comparison surface in a differential test.	Enumerated explicitly per court so “we tested it” always answers “tested what, exactly?”
Raw capture	Golden file or VCR-style recorded interaction.	Immutable by rule; normalization creates a new derived artifact and never overwrites the raw one.
Residual	A failing diff in golden-master or snapshot testing.	Not auto-accepted or auto-updated on mismatch; it gets identity, lifecycle, and required disposition before closure.
Structured Unknown	Flaky test or skipped test.	Explicit third state distinct from pass/fail, carrying full residual metadata; flaky becomes a typed, routable object.
Disposition	Triage label on a bug ticket.	Required reason string enforced at the tool level; no silent default disposition.
Residual token / endoduction	Bug classification or defect-taxonomy tagging.	Deterministic function rather than human judgment call; the same residual coarsens to the same token under the same schema.
Receipt	SLSA-style provenance or build attestation.	Applied to behavioral test outcomes, not only supply-chain artifacts.
Claim compiler	No direct standard equivalent.	Prose claims are generated from receipts and mechanically refused when evidence does not cover them.
Non-claim / negative capability	Known limitations section in a README.	Structurally paired with each positive claim rather than optional prose at the bottom.
Invariant bank	Regression test suite.	Explicitly linked back to the motivating residual, with a required negative control.
Minimization	Test-case reduction in the C-Reduce lineage.	Required before a residual can be marked resolved when practical.
Negative control / mutation operator	Mutation testing.	Applied to invariants specifically, not code coverage generally: does this court reject the seeded failure?
Normalization, governed	Ignoring timestamps or UUIDs in a snapshot diff.	Requires the normalizer itself to be versioned evidence, and the resulting claim excludes the normalized field.
Authority separation	Multiple sources of truth in a data-consistency system.	Applied to test oracles; an RFC and a legacy binary can disagree, and that conflict remains visible.
Semantic non-bypass rule	No direct standard equivalent.	The hard form of “do not let the README outrun the tests,” enforced as a code-path constraint rather than a review habit.

One-line summary for a skeptical senior engineer. FRF is differential testing plus golden-master discipline plus SLSA-style evidence receipts, with a claim compiler that refuses to let documentation say more than the receipts prove.

C.3 Part 2: The Operating Core

Everything below is the practical payload of FRF. If a developer internalizes this page, they can run the framework without the rest of the paper.

Central rule.

A mismatch is not noise until evidence establishes why it is noise.

Kernel: ten steps, in order.

1. Admit the authority.
2. Declare the claim you are trying to earn.
3. Name the court: one narrow question.
4. Capture raw observations from both sides.
5. Preserve residuals before interpretation.
6. Coarsen residuals into typed tokens.
7. Route tokens to the next court or blocker.
8. Add negative controls.
9. Emit receipts.
10. Compile claims and non-claims; do not hand-write either when they can be generated.

Canonical loop.

Authority → Court → Capture → Residual → Endoduction
→ Route → Disposition → Receipt → Claim.

Twelve operating rules.

1. A green test without a bounded claim is a local signal, not an argument.
2. Compatibility is a property of code observed against an admitted authority, not of code alone.
3. The authority is not sacred. The observation is.
4. Preserve raw residuals before interpreting, normalizing, filtering, or fixing them.
5. Treat every normalizer as claim-weakening machinery until proven otherwise.
6. Never call an unmeasured surface compatible.
7. Never let a fixture count substitute for surface coverage.
8. Never delete a residual without a disposition.
9. Never convert an intentional divergence into a parity claim.
10. Never let the README outrun the receipts.
11. If a statement can be generated from evidence, generate it.
12. A receipt is the build artifact of a claim.

Residual states. Use all of these states rather than collapsing the workflow into pass/fail:

```
admissible | boundary | violation | recovery | unknown |
intentional_divergence
```

Minimal residual token.

```
kind: text | exit | wire | state | timing | filesystem |
      security | platform | performance | harness | unknown
surface: string
authority: string
disposition: open | fixed | intentional | environmental |
              oracle_version | harness | unknown
next_court: string
```

A disposition is not valid without a one-line reason attached to it.

C.4 Worked Example

Question. For malformed input, does the candidate preserve the reference’s exit class and first diagnostic line?

Raw capture.

```
reference.exit: 2
reference.stderr[0]: "tool: file:4: unknown directive 'servre'"
candidate.exit: 1
candidate.stderr[0]: "error: unknown directive servre at line 4"
```

Initial residuals. There are two residuals, both open. Do not decide which side is “right” yet.

Endoduction. The exit residual routes to exit-class minimization. The text residual routes to diagnostic-wording minimization. Both currently block any compatibility claim.

Minimization. The fixture reduces to a single misspelled directive. That alone is sufficient to reproduce both residuals.

Resolution. The candidate is patched to match the exit class. Diagnostic wording is kept different intentionally for clarity.

Dispositions.

```
exit -> fixed
text -> intentional-divergence
      reason: clearer wording, documented
```

Claim compiler may say.

For reference `ref-cli-1.8.2`, fixture family X , and environment E , the candidate preserves malformed-input exit class for the minimized directive-error cases.

Claim compiler must refuse.

The candidate is byte-identical on stderr, fully CLI-compatible, or a drop-in replacement for all malformed-input behavior.

That refusal is not administrative caution. It is the artifact FRF exists to produce.

C.5 When Not To Use This

Not every project needs byte-exact historical compatibility. If there is no meaningful external authority to admit against, such as a reference implementation, contract, protocol, or prior release, conventional specification-driven development is probably the better tool. FRF earns its cost where a claim depends on an admitted authority.

References

- [1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. DOI: 10.1109/TSE.2014.2372785.
- [2] W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- [3] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2011.
- [4] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2012.
- [5] R. Birkner, T. Brodmann, P. Tsankov, L. Vanbever, and M. Vechev. Metha: Network verifiers need to be correct too! In *18th USENIX Symposium on Networked Systems Design and Implementation*, pages 99–113, 2021.
- [6] **infinityabundance/chrony-rs**: forensic Rust reconstruction of chrony time-discipline behavior. Repository evidence reviewed 20 August 2026, including oracle, parity, generated-documentation, negative-capability, and receipt documentation. github.com/infinityabundance/chrony-rs.
- [7] **infinityabundance/m4-rs**. Repository evidence reviewed 20 August 2026, including casefile, source-audit, and oracle-profile documentation. github.com/infinityabundance/m4-rs.
- [8] **infinityabundance/openntpd-rs**: clean-room forensic Rust reconstruction of OpenNTPD 7.9p1. Repository evidence reviewed 20 August 2026, including README, archaeology, generated-documentation, no-original-code, and parity documentation. github.com/infinityabundance/openntpd-rs.
- [9] **infinityabundance/ntpsec-rs**: forensic Rust reconstruction of NTPsec. Repository evidence reviewed 20 August 2026, including README, courts, generated-documentation, source-archaeology, and parity documentation. github.com/infinityabundance/ntpsec-rs.
- [10] **infinityabundance/zic-rs**: memory-safe Rust IANA zic/TZif compiler. Repository evidence reviewed 20 August 2026, including trust ladder, vendor-oracle, operational parity, release-ladder, audit, and non-claim documentation. github.com/infinityabundance/zic-rs.
- [11] **infinityabundance/gnucobol-rs**: native Rust forensic implementation of GnuCOBOL behavior. Repository evidence reviewed 20 August 2026, including admitted corpus, claim ladder, case files, oracle profile, byte-level runtime parity, and non-claim documentation. github.com/infinityabundance/gnucobol-rs.
- [12] **infinityabundance/ncurses-native**: Rust reproduction of observable ncurses terminal byte output for a bounded terminal/build surface. Repository evidence reviewed 20 August 2026, including byte-output parity claims, clean-room statement, and explicit non-claims. github.com/infinityabundance/ncurses-native.
- [13] **infinityabundance/bind9-rs**. Repository evidence reviewed 20 August 2026, including forensic library, residual, court, receipt, and parity-ledger documentation. github.com/infinityabundance/bind9-rs.

- [14] R. de Beer. *Drift-Slew Fusion Bootstrap: A Deterministic Residual-Based State Correction Framework*. Zenodo, 2026. doi:10.5281/zenodo.18706455.
- [15] R. de Beer. *Residual Trajectories as Primary Epistemic Objects: Drift-Slew Decomposition of Laplacian Relaxation and the Spectral Fingerprint of Graph Structure*. Zenodo, 2026. doi:10.5281/zenodo.19382012.
- [16] R. de Beer. *Alternative Deterministic Structural Inference: The DSFB Stack for Reconstruction, Causal Architecture, Trust Recursion, and Historical Replay*. Zenodo, 2026. doi:10.5281/zenodo.19028440.
- [17] R. de Beer. *Deterministic Disturbance Modeling Framework for Residual-Envelope Fusion Systems*. Zenodo, 2026. doi:10.5281/zenodo.18806150.
- [18] R. de Beer. *DSFB-Debug Structural Detector-Field Residual Semiotics Engine for Software Debugging: A Deterministic Augmentation Layer for Typed Residual Interpretation of Execution Traces, Log Streams, and Observability Telemetry in Production Software Systems*. GitHub repository README and Zenodo software record, 2026. github.com/infinityabundance/dsfb/tree/main/crates/dsfb-debug; doi:10.5281/zenodo.20088863.
- [19] R. de Beer. **dsfb-forensics**: reference specification and audit layer for the Drift-Slew Fusion Bootstrap stack. GitHub repository README, 2026. github.com/infinityabundance/dsfb/tree/main/crates/dsfb-forensics.
- [20] R. de Beer. *DSFB-Database: A Deterministic, Read-Only Structural Observer for Residual Trajectories in SQL Database Telemetry*. GitHub repository README and Zenodo software record, 2026. github.com/infinityabundance/dsfb/tree/main/crates/dsfb-database; doi:10.5281/zenodo.19656368.
- [21] R. de Beer. *DSFB Structural Semiotics Engine for Deterministic Rust Crate Auditing*. GitHub repository README and Zenodo software record, 2026. github.com/infinityabundance/dsfb/tree/main/crates/dsfb-gray; doi:10.5281/zenodo.19600872.
- [22] R. de Beer. *DSFB-GPU: Clear-Box Pure Deterministic Inference CUDA Acceleration for Replayable Trace-Event Verdicts*. Zenodo and GitHub repository, 2026. doi:10.5281/zenodo.20346478.
- [23] R. de Beer. DSFB-Debug Zenodo record. zenodo.org/records/20088863, 2026.
- [24] R. de Beer. DSFB prior-art Zenodo DOI record. doi:10.5281/zenodo.19446580, 2026.
- [25] R. de Beer. *Slew-Aware Trust-Adaptive Nonlinear State Estimation for Oscillatory Systems With Drift and Corruption*. Zenodo record. zenodo.org/records/18642887, 2026.
- [26] R. de Beer. *Drift-Slew Fusion Bootstrap: A Deterministic Residual-Based State Correction Framework*. zenodo.org/records/18706455, 2026.
- [27] R. de Beer. *Hierarchical Residual-Envelope Trust: A Deterministic Framework for Grouped Multi-Sensor Fusion*. zenodo.org/records/18783283, 2026.
- [28] R. de Beer. *Alternative Deterministic Structural Inference: The DSFB Stack for Reconstruction, Causal Architecture, Trust Recursion, and Historical Replay*. zenodo.org/records/19028440, 2026.

- [29] R. de Beer. *Residual Streams in Gradient Flows: Drift-Slew Decomposition of Optimization Residuals as a Convergence Diagnostic and Saddle-Proximity Indicator*. zenodo.org/records/19410569, 2026.
- [30] R. de Beer. *Residual Trajectories as Primary Epistemic Objects: Drift-Slew Decomposition of Laplacian Relaxation and the Spectral Fingerprint of Graph Structure*. zenodo.org/records/19382012, 2026.
- [31] R. de Beer. *DSFB Structural Semiotics Engine: A Survey of Applications Across Safety-Critical Domains*. zenodo.org/records/19204366, 2026.